

面向多模态推理的高效长上下文建模

庄博涵

PI of ZIP Lab @ Zhejiang University

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon 上海站
全球软件开发大会

大模型正在重新定义软件
Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon 北京站
全球人工智能开发与应用大会

探索 AI 应用边界
Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

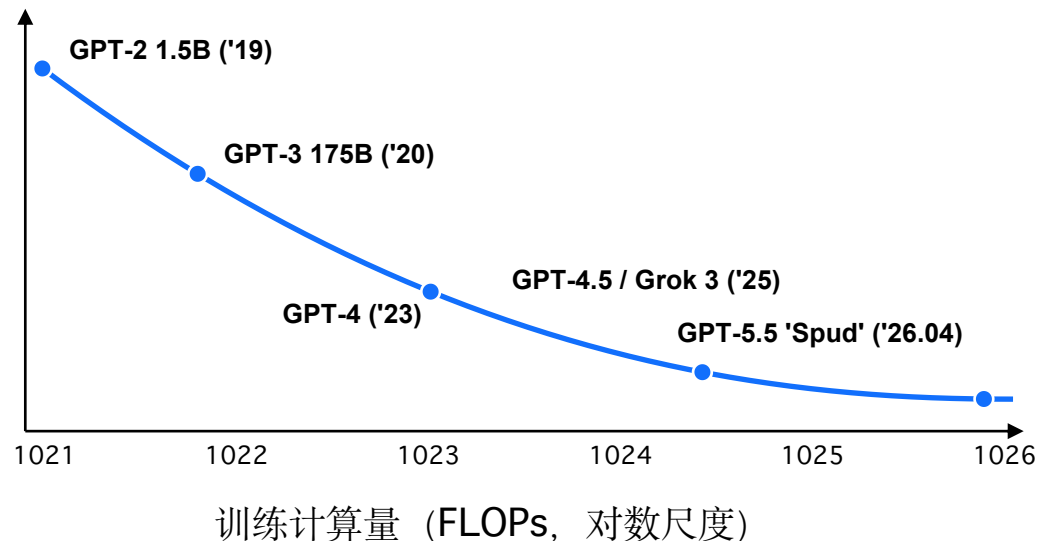
· 大模型安全

Part 1: Background of Efficiency & Agent

Scaling Law 的叙事依然在继续

训练时的 Scaling Law

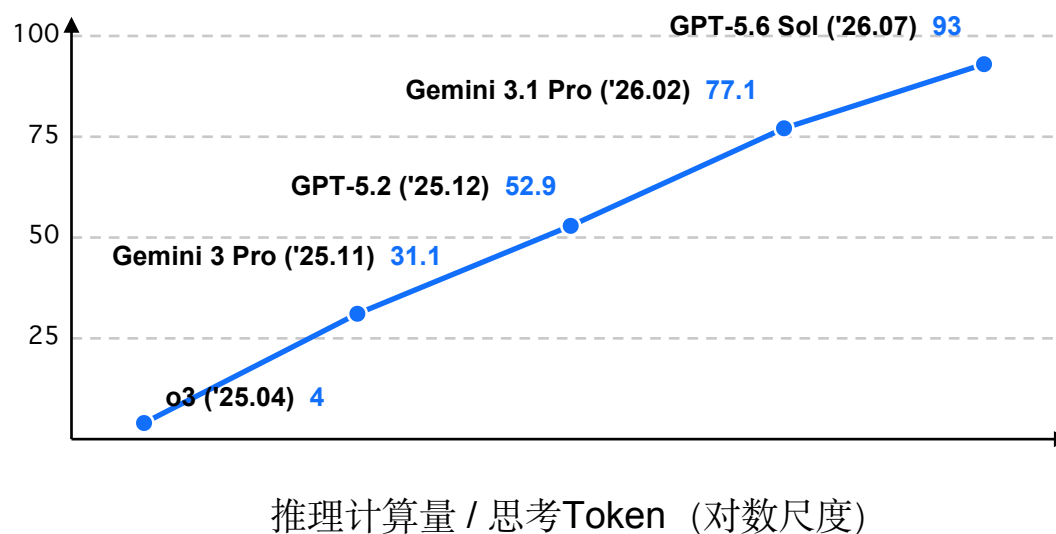
预训练损失 (对数尺度)



训练时：规模越大，损失越低；合成数据90%以上

推理时的 Scaling Law

准确率 (%)



推理时：思考越多，准确率越高；部署成本也会上升

Agentic AI 技术演进：从工具调用到自主智能体



L1-L2 已成熟；当前产品能力集中在 L3-L4，向 L5 自主智能体演进的趋势较为确定

■ 双层优化(Bi-Level Optimization): Harness 与 Model

Agent = Model + Harness

$$\begin{aligned} \max_{\theta} \quad & J(\theta, h^*(\theta)) \\ \text{s.t.} \quad & h^*(\theta) = \arg \max_h J(\theta, h) \end{aligned}$$

θ

black-box model 参数 慢 — 隔很久才 update 一次

h

white-box harness 配置 快 — 频繁迭代

$J(\theta, h)$

部署后的 system 在任务上的期望 reward 优化目标

异步实现: harness 快, model 慢

Inner loop · harness

快

每一步: $h \leftarrow \arg \max_h J(\theta, h)$

θ 冻结, 反复运行直至收敛



Outer loop · model

慢

每 $T \gg 1$ 步: $\theta \leftarrow \theta + \eta \nabla_{\theta} J(\theta, h)$

间隔较久才更新一次

Harness (白盒) 优化快速且低成本; 模型 (黑盒) 更新缓慢且昂贵。

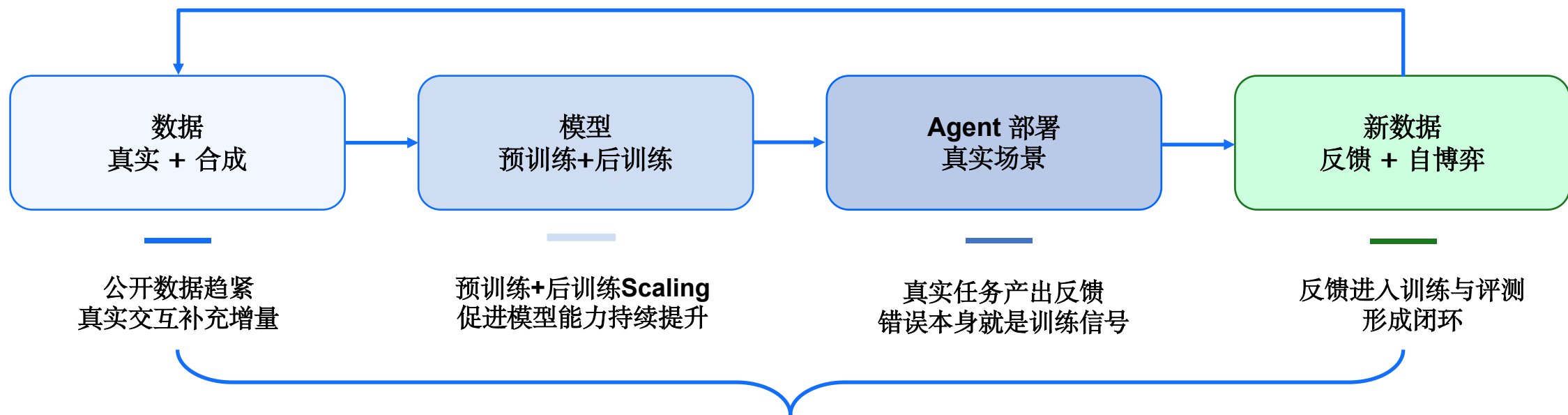
内层优化问题: Harness 自我进化

Harness 是模型控制系统, 决定模型看到什么、能做什么、怎么走、记住什么、以及如何自查



- **Codex / Claude Code / OpenCode**: 声明式插件, Markdown skill + MCP 配置, 改完重启即可
- **DeepSeek DSH**: 命令式插件, agent loop 本身也是插件, 可运行时热替换 (Cordis 管撤销与依赖)
- **上限相似、下限不同**: DSH 的结构优势在于让 harness 自进化

外层优化问题: Model自进化的数据飞轮



预训练、真实推理和后训练相互叠加，形成复合效应，推动能力加速增长。

从 LLM 到 Agent: 效率问题的转变

LLM 时代问:
「模型能不能做到?」

Agent 时代问:
「做到一次要花多少时间和钱?」

long-horizon 单任务几十轮不中断

multi-turn 上下文逐轮累积、反复重读

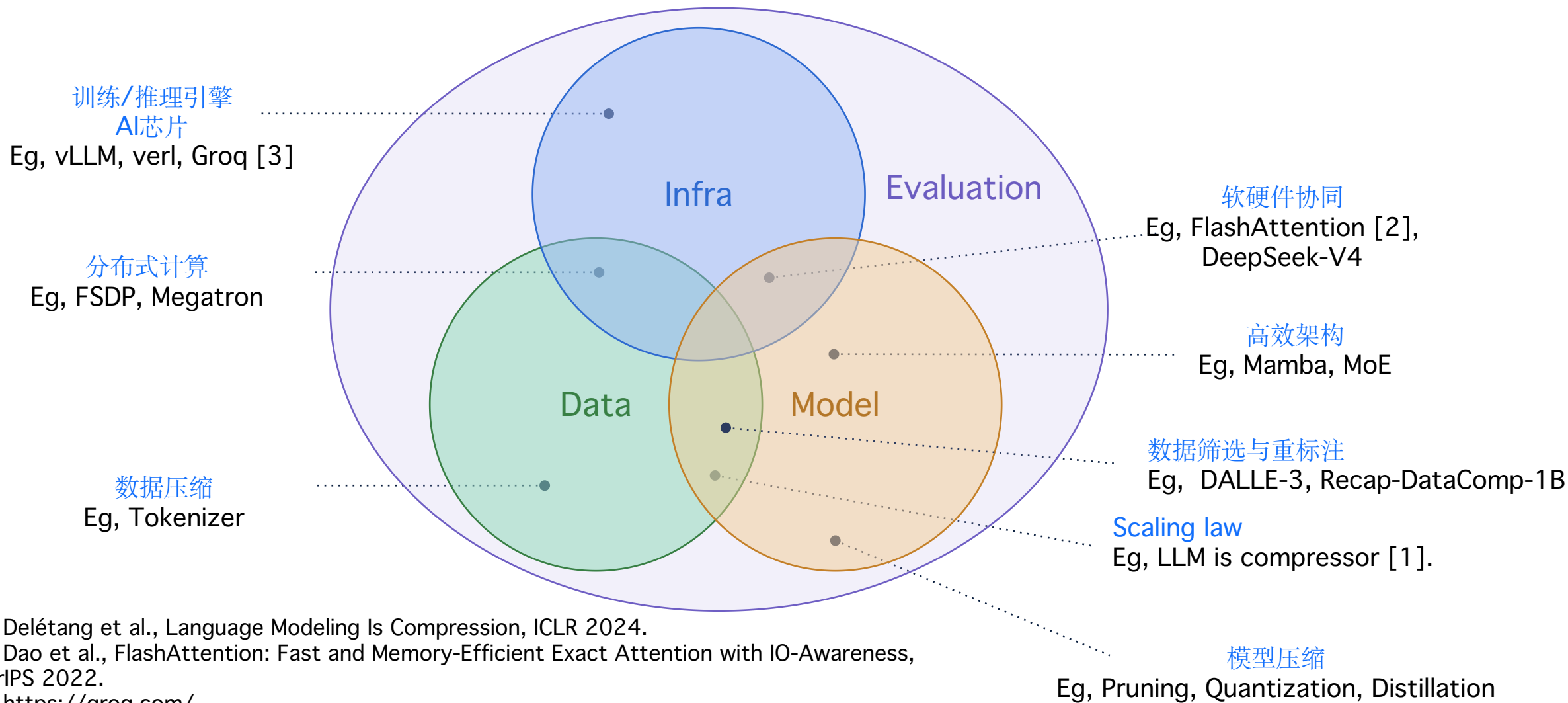
multimodal 视频帧 / 音频 token 密度极高

三者叠加 → 单次任务的推理开销比单轮问答高出 2-3 个数量级

指标	单轮 LLM 问答	一次分钟级长视频生成 Agent 任务	差距
任务构成	1 次 prompt → 1 次回答	1 段脚本 → 12 个镜头 × 5 s + 逐镜头一致性校验	—
推理轮次	1 turn	40-60 turns (镜头规划 / 重绘 / 对齐)	~50x
上下文峰值	~1 K tokens	~300 K tokens (参考帧 + 历史镜头)	~300x
累计处理 token	$\sim 1 \times 10^3$	$\sim 5 \times 10^7$ (1440 帧逐帧展开)	~5万x
端到端延迟	2-5 s	5-15 min	~150x
单次成本	≈ \$0.005	≈ \$3-8	~1000x

便宜又能干活的 token 产得越多, 越赚钱——agent 和多模态生成就是典型。

效率优化的四个因素

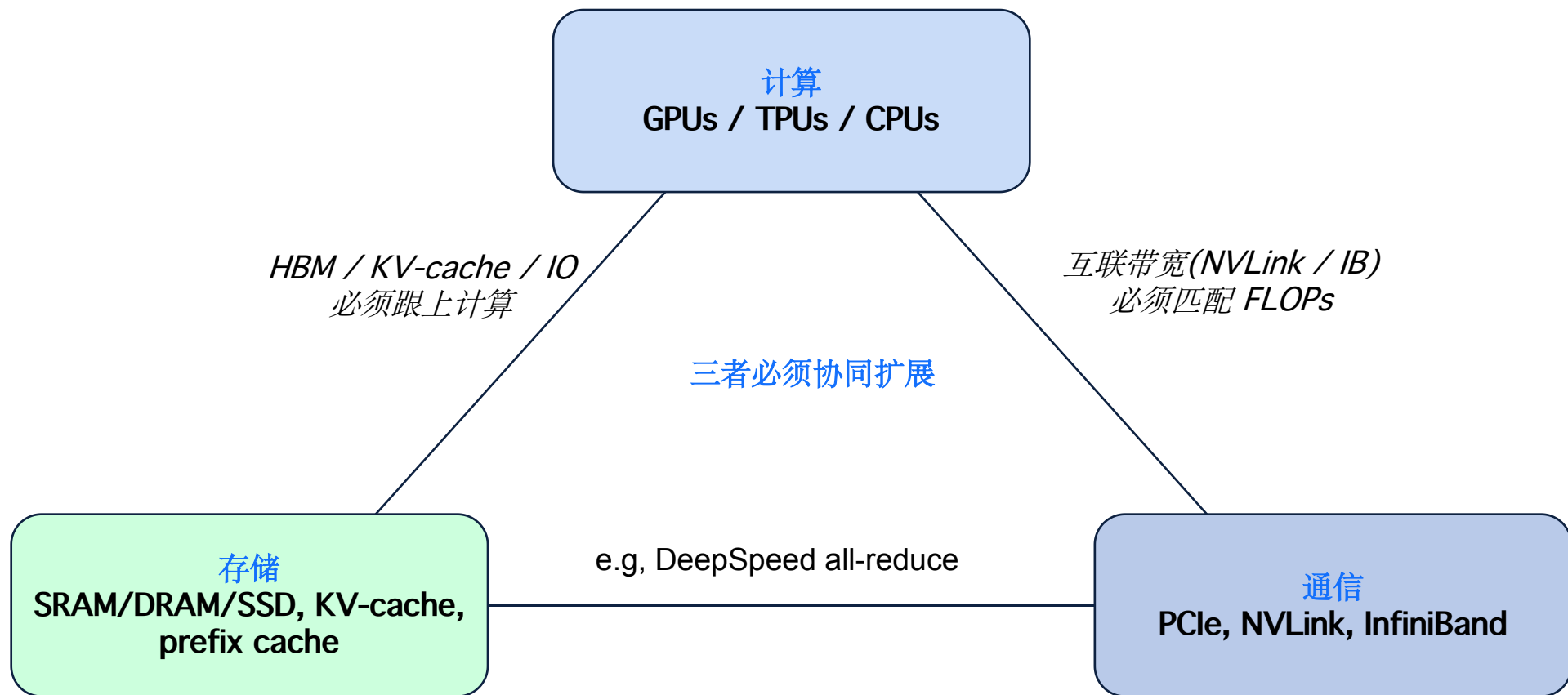


[1]: Delétang et al., Language Modeling Is Compression, ICLR 2024.

[2]: Dao et al., FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, NeurIPS 2022.

[3]: <https://groq.com/>

机器学习系统的三角约束：计算、存储与通信



任一环节成为瓶颈，整体系统效率都会被拉低

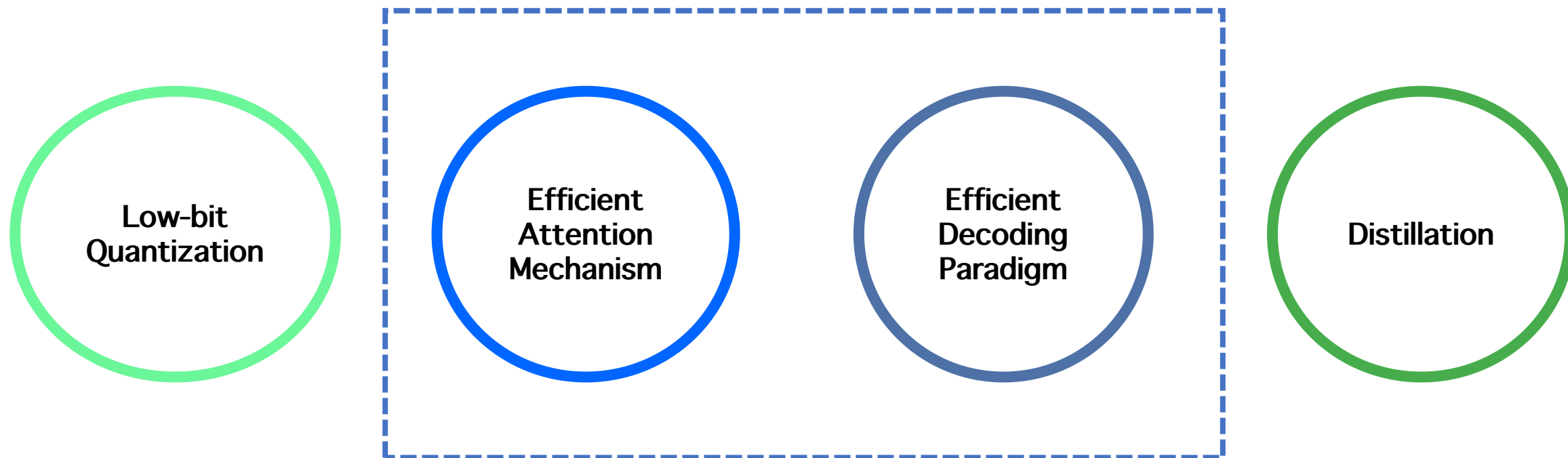
如何构建高效、自进化的多模态Agent?



Part 2: Efficient Multimodal Inference

第一层：引擎层 — 通用高效推理

Efficient Inference 总览

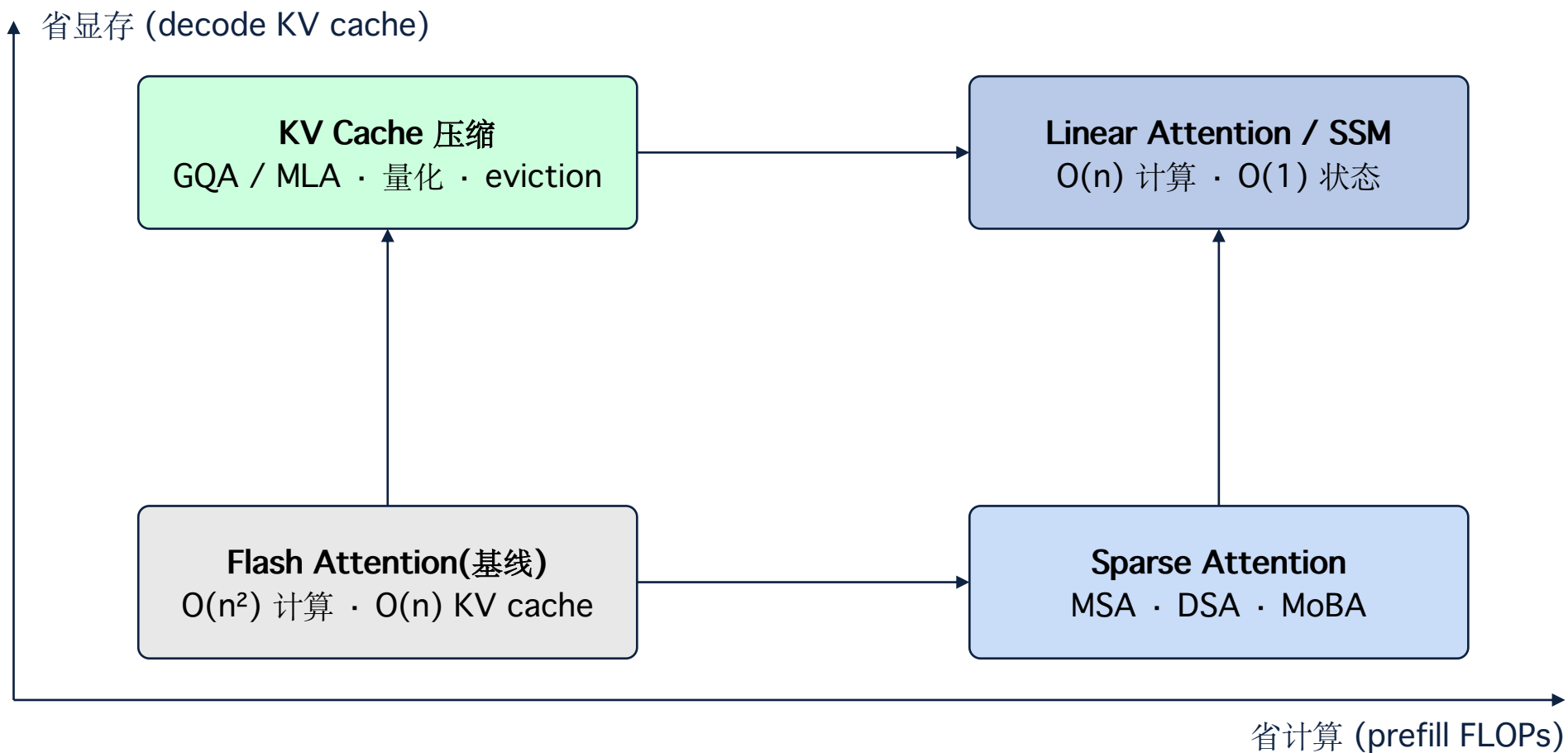


Principle: Algorithm and Infra Co-design



Efficient Attention

Efficient Attention: 两个维度的全景图



Linear attention 同时把两个轴推到极限; hybrid 架构在质量与效率之间插值

FlashAttention: IO 感知的精确 Attention

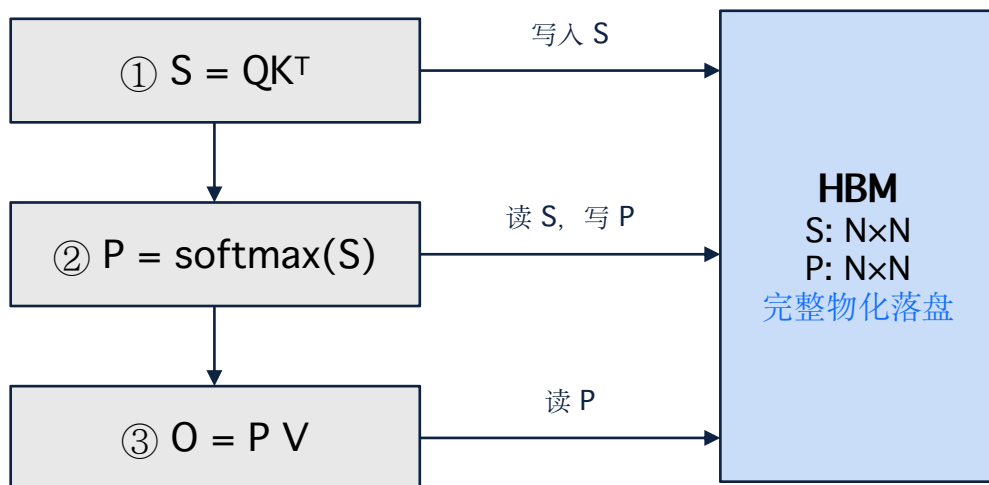
SRAM (片上) ≈ 19
TB/s $\cdot$ 20 MB

带宽差
 $\sim 10\times$

HBM $\approx 1.5\text{--}2$ TB/s $\cdot$ 40 GB

大矩阵反复读写 HBM $\rightarrow$ attention 属于 **IO-bound**

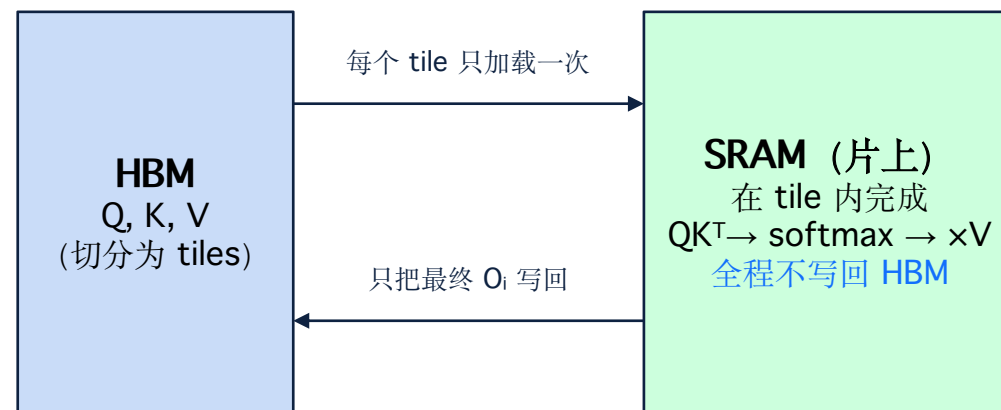
标准 attention: 中间结果在 HBM 之间来回搬运



S、P 的访存量为 $O(N^2)$, 远高于 Q/K/V 的 $O(Nd)$ ($N \gg d$)

$\rightarrow$ 时间都花在 HBM 搬数据, 而不是算力

FlashAttention: 分块 tiling, 把计算留在 SRAM 内



$N \times N$ 的 S、P **从不完整写入 HBM**; 只有 tile 大小的中间结果短暂存活在 SRAM

HBM 访问次数大幅减少!

[1] Dao et al., FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, NeurIPS 2022.

Linear Attention

回顾一下: Softmax Attention Linear Attention

$$O = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

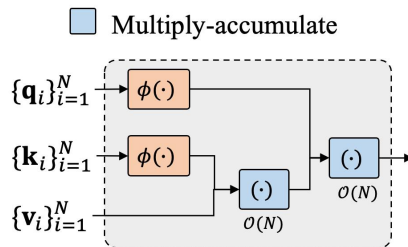
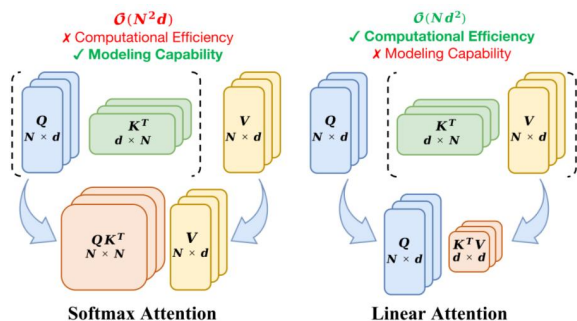
$$O = \phi(Q)(\phi(K)^\top V)$$

线性计算复杂度:

- 通过重排矩阵乘法, 使计算复杂度变为线性
- 所有历史压缩成一个固定大小的历史状态

$$S_t = S_{t-1} + k_t v_t^\top$$

$$O_t = q_t S_t$$



Linear Attention 的多种变

✦

$$S_t = \alpha_t S_{t-1} + k_t v_t^\top, \alpha_t \in [0, 1]$$

Gated Linear Attention (GLA)

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t k_t v_t^\top$$

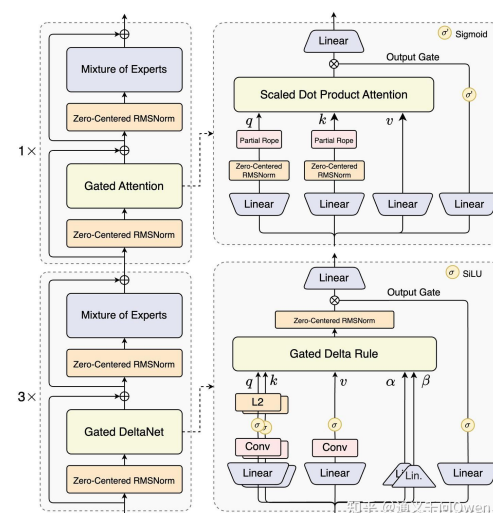
$$\alpha_t, \beta_t \in [0, 1]$$

Gated Delta Net (GDN)

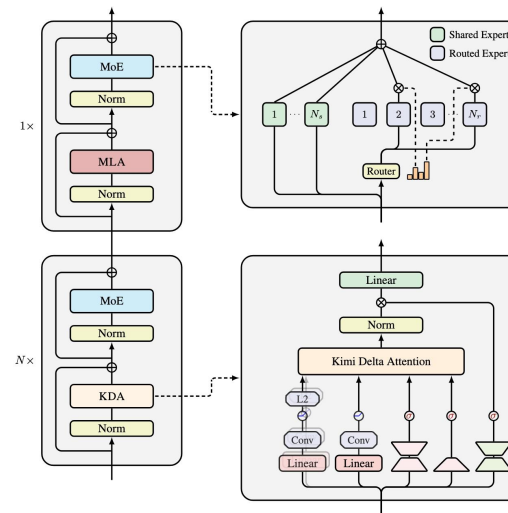
$$S_t = \text{diag}(\alpha_t) S_t (I - \beta_t k_t k_t^\top) + \beta_t k_t v_t^\top$$

KDA

Hybrid Architecture



Qwen 3 Next



Kimi K3

Efficiency 上的挑战

- Prefix Cache 命中率, 兼容性 (vllm/sglang) 与高效管理
- 串行递推的高效执行

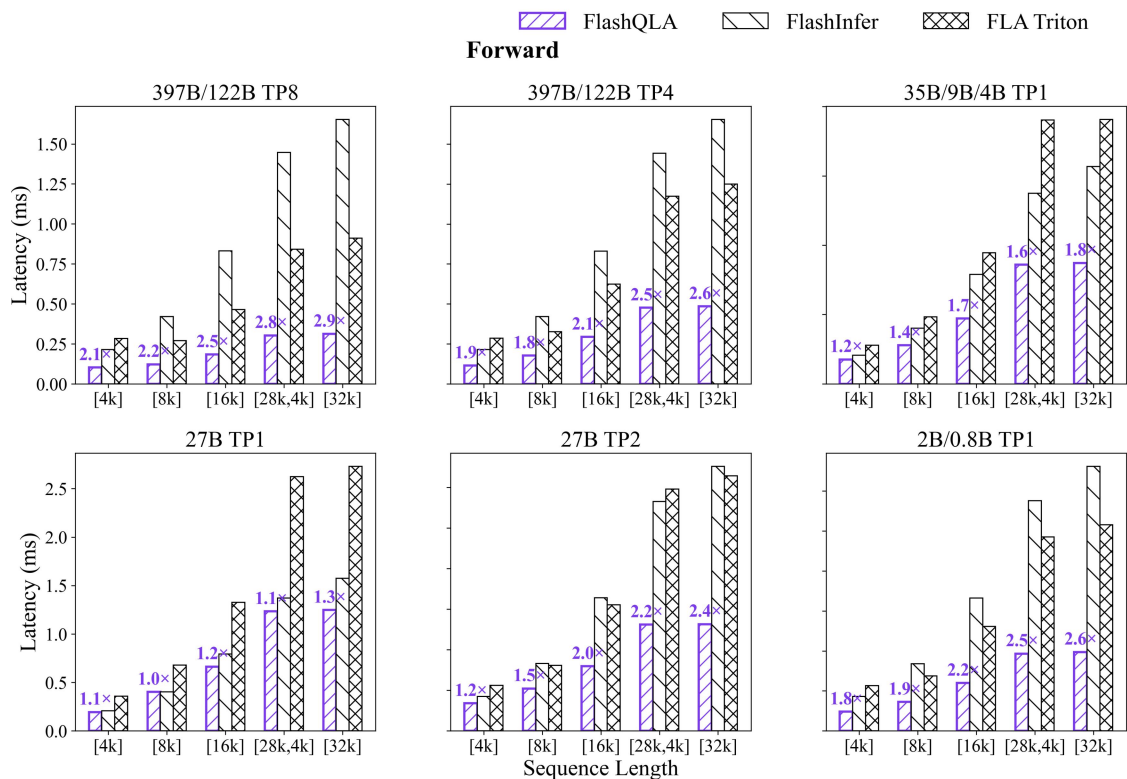
FlashQLA

CP- and Bwd-Friendly Fused Linear Attention Kernels for GDN

- 门控感知的自适应上下文并行
 - 针对 TP 和长序列场景
 - 根据工作负载进行自适应 CP 切分 (intra-card)
 - 利用 Gate 的误差衰减减小 CP Overhead
- 硬件友好的 GDN 重构
 - Gate Decay 外提
 - 降低 Tensor Core / CUDA Core / SFU 开销
- Warp-Specialized TileLang 融合 Kernel
 - Warpgroup Specialization
 - Overlap数据搬运与计算

<https://github.com/QwenLM/FlashQLA>
<https://qwen.ai/blog?id=flashqla>

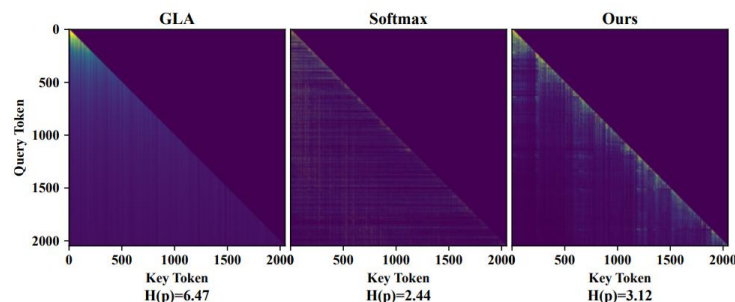
- 前向: 2-3x Speedup
- 反向: 2x Speedup



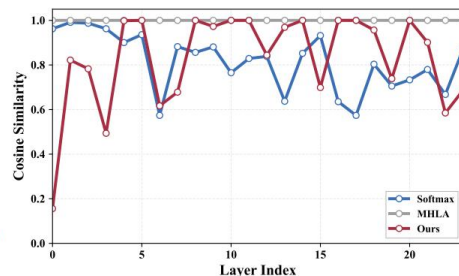
HLA: Expressive Hybrid Linear Attention via Chunk-wise Softmax Mixing

Zhuokun Chen, Xi Lin, Xiyu Wu, Jiahao He, Jianfei Cai, Bohan Zhuang

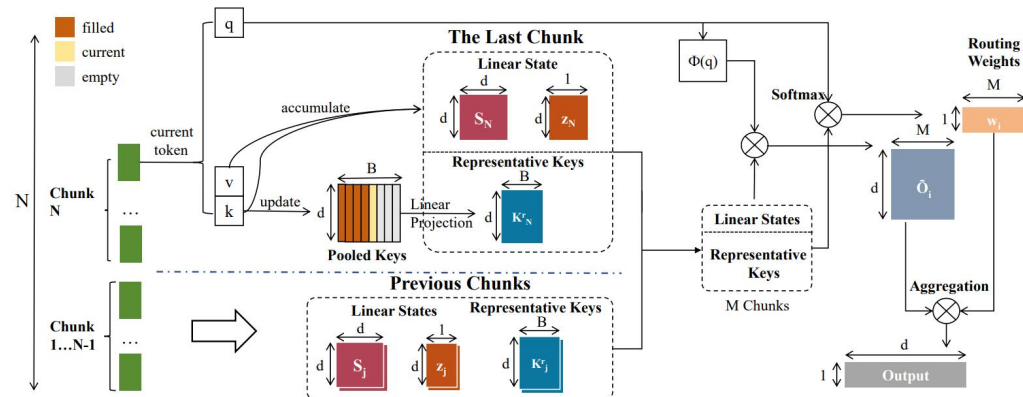
研究动机



(a) Attention-score and entropy comparison among GLA, softmax attention, and HLA. (b) Cross-sample chunk-level similarity during decoding.



研究方法



- 表达能力: 单一线性状态难以保留全部历史 → 每个 chunk 独立存储状态
- 查询相关检索: 不同 query 依赖不同历史 chunk → 按需选择, 而非平均

HLA: 切分上下文 → 每个 chunk 保存线性状态 + 路由 key → 根据 query 对 chunk 进行 Softmax 路由 → 聚合选中的 chunk 状态

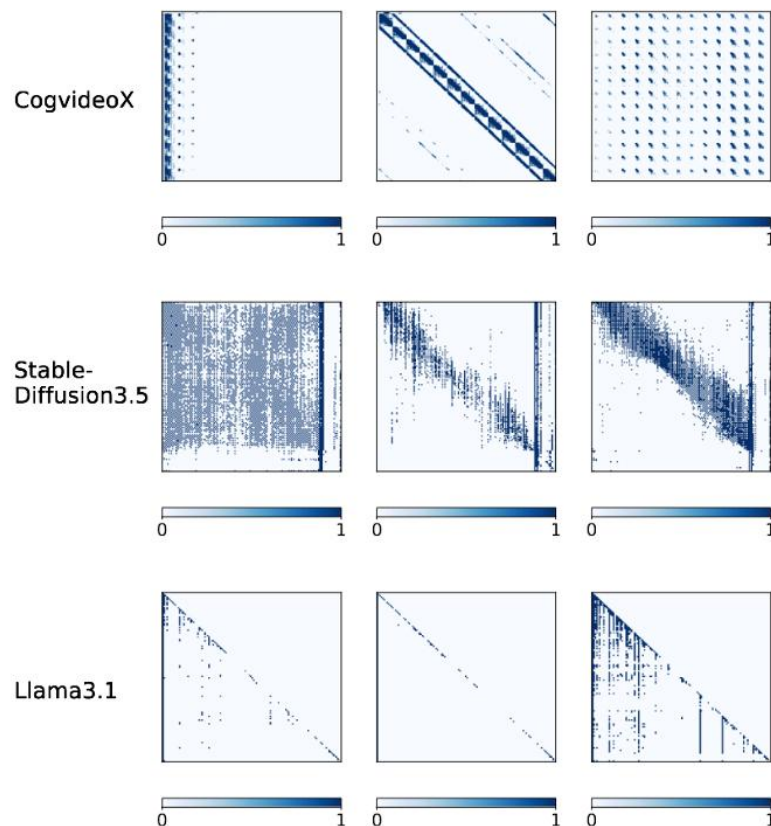
Model	2WM	HQA	Mus	QQA	NQA	SSM	TQA	PEN	PZH	QMS	GvR	MNs	RBP	LCC	Avg.
GDN-1.3B	2.37	2.16	1.47	2.80	0.74	25.35	19.67	0.50	0.75	11.63	7.43	7.78	52.69	59.09	13.89
MHLA-1.3B	2.30	1.91	2.14	3.50	0.67	21.04	19.64	0.00	0.75	10.72	7.44	8.02	52.03	58.52	13.48
HLA-1.3B	1.49	2.06	1.81	3.15	0.64	21.16	29.43	0.25	1.44	12.07	8.55	8.49	53.05	59.26	14.49

LongBench **Avg 14.49** vs GDN 13.89 / MHLA 13.48; 提升主要集中在检索任务上 — TriviaQA **29.43** vs 19.67 / 19.64.

目标: 保持线性注意力的计算成本, 同时恢复 Softmax 级别的选择性检索能力。

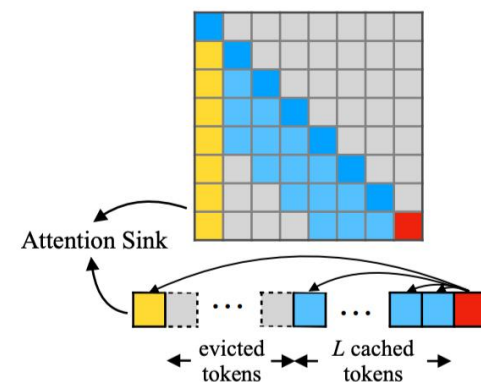
Sparse Attention

Attention Map天然具有高稀疏性
注意力分数集中在一小部分token上



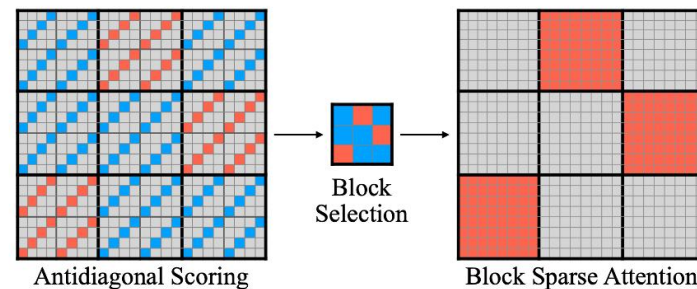
静态稀疏注意力

- 基于固定的稀疏模式
- StreamingLLM, MInference, ...



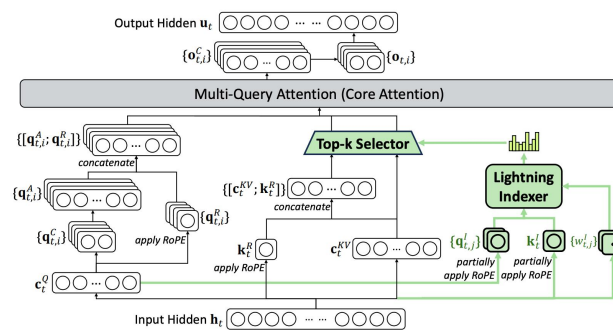
动态块级稀疏注意力

- 动态选择需要计算的块
- SpargeAttn, XAttention, ...

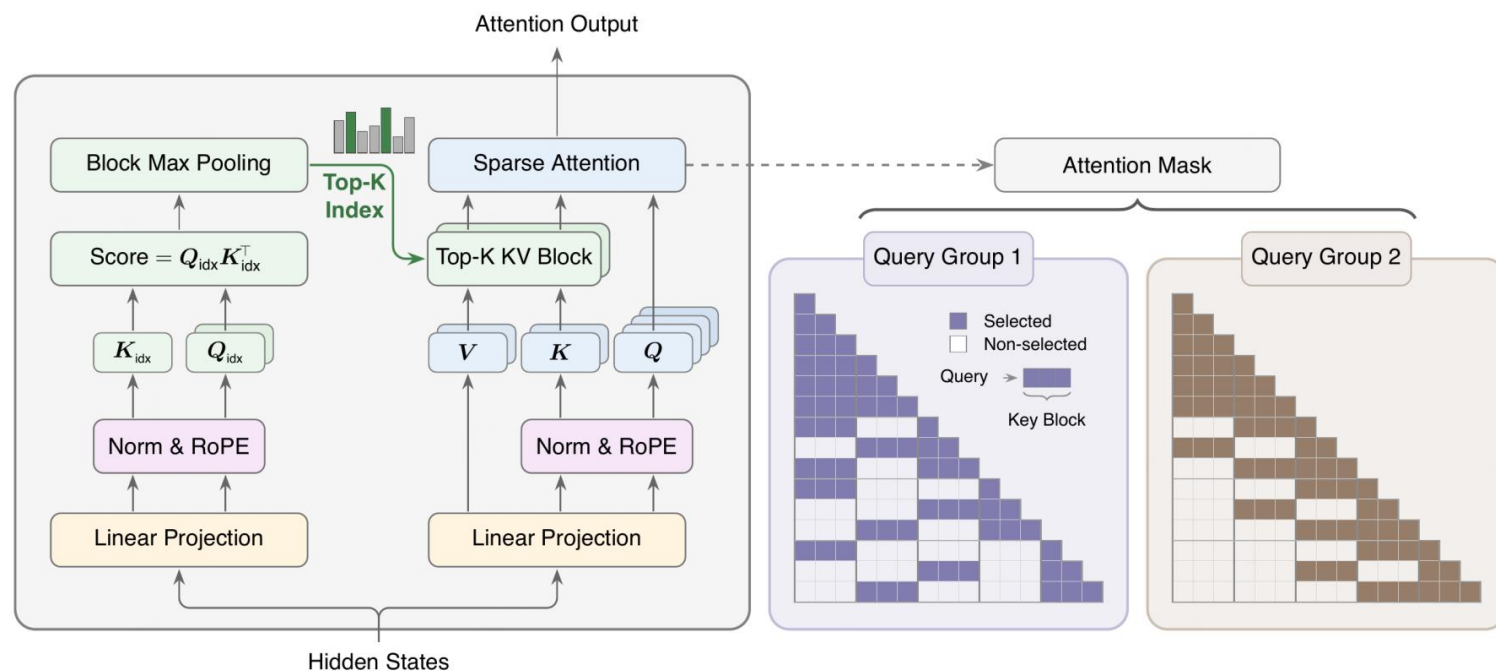


动态token级稀疏注意力

- 细粒度地选择需要计算的token
- DeepSeek Sparse Attention, ...



MiniMax Sparse Attention



MSA vs. DSA

机制与取舍

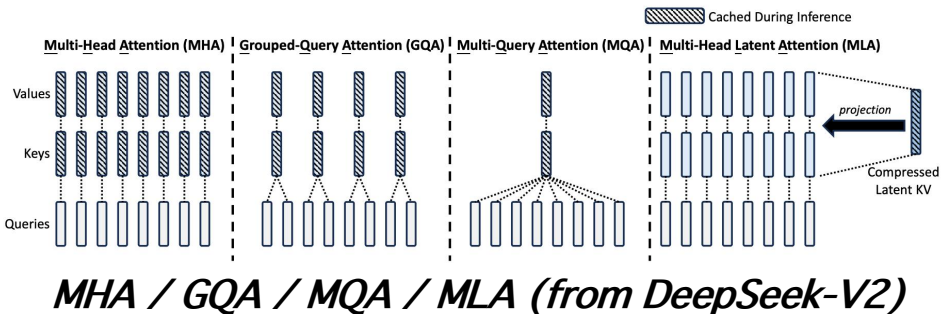
	MSA	DSA
选择单元	块 Block 更粗	Token 更细
Top-K 排序	排序块更少 成本更低	排序 Token 多 成本更高
Top-K 集合	按查询组共享 更细	所有头共享 更粗
KV 基础	基于 GQA 分组共享KV	基于 MQA 单组共享KV

同等算力下

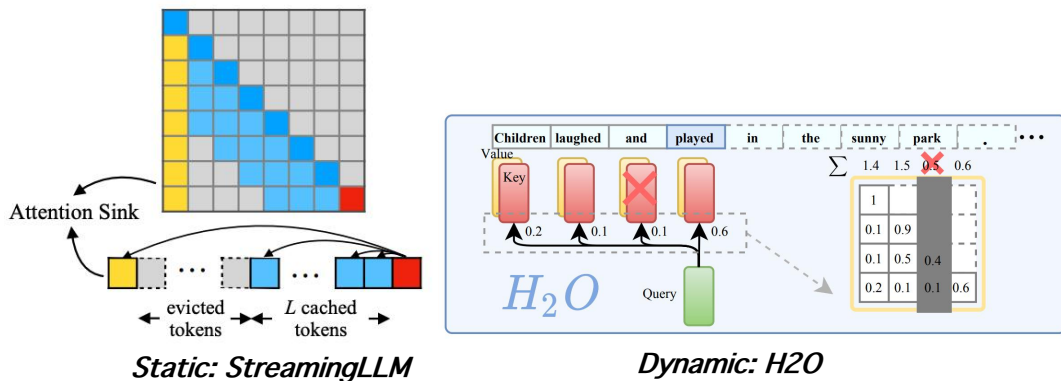
MSA: Top-K 集合更多, KV 覆盖更广

DSA: Token 级选择更精细

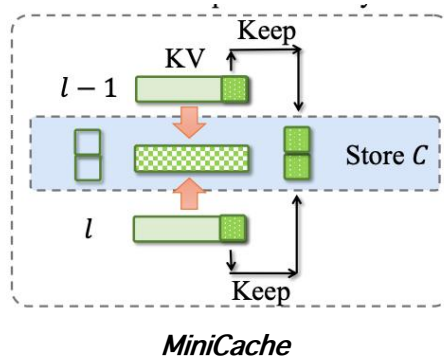
架构设计——共享 / 低秩 KV heads



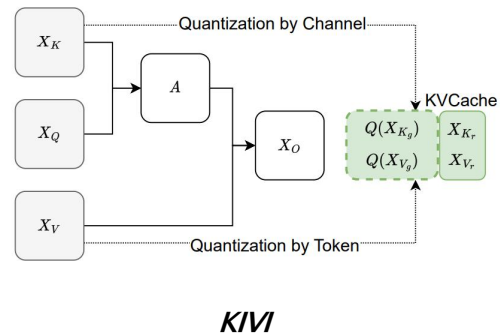
Token 剪枝 (静态 & 动态)



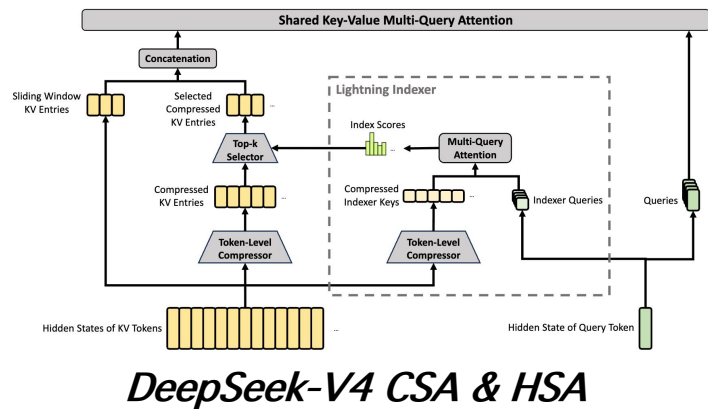
跨层合并



KV 量化



基于学习的 Token 压缩



另一种极端方案: **linear attention / Test-time Training** 将 KV state 保持为固定大小——压缩率最高, 但牺牲精确检索能力

[1] Xiao et al., StreamingLLM, ICLR 2024. [2] Zhang et al., H2O, NeurIPS 2023. [3] Liu et al., KIVI, ICML 2024. [4] Liu et al., MiniCache, NeurIPS 2024. [5] Ainslie et al., GQA, EMNLP 2023.

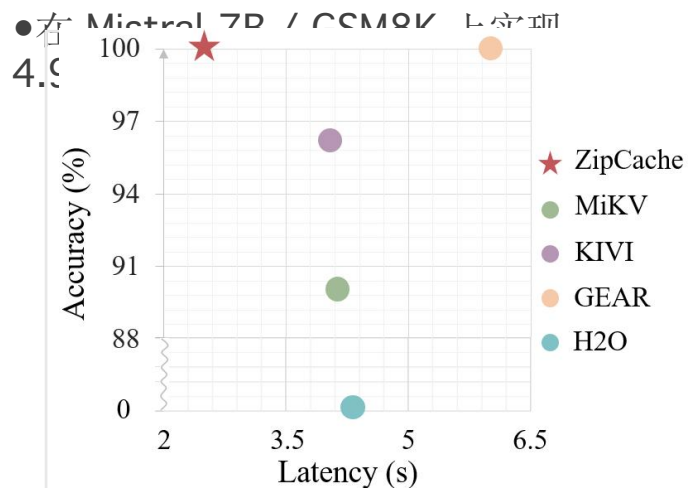
[6] DeepSeek-AI, DeepSeek-V2 / V4.

Lab 的一些 KV Cache 压缩方法

ZipCache

NeurIPS 2024

- KV Cache 量化可能造成准确率损失，需要根据 Token 重要性采用不同 bit-width
- 用归一化 Attention 权重估计 Token 重要性；重要 Token 使用 4-bit，其余 Token 使用 2-bit，并兼容 FlashAttention

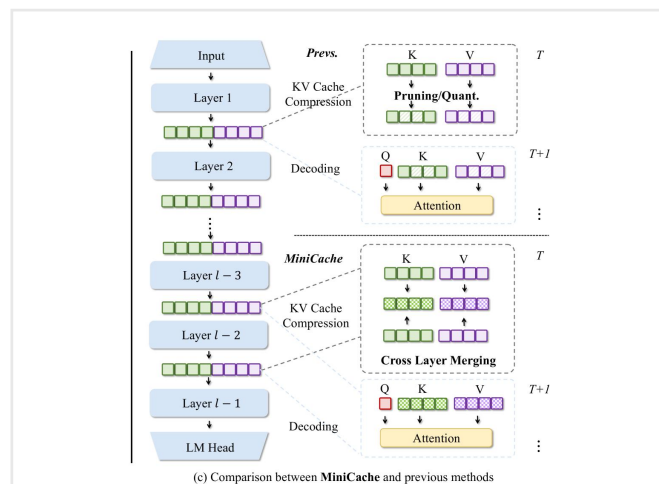


与已有 KV 量化方法的准确率-延迟对比

MiniCache

NeurIPS 2024

- 相邻中深层的 KV 状态高度相似，可去除 depth 维度冗余
- 跨层合并 KV 状态：插值方向、保留幅值；离群 Token 不参与合并
- KV Cache 内存占用最多降低 41%，相较 FP16 吞吐量提升约 5x

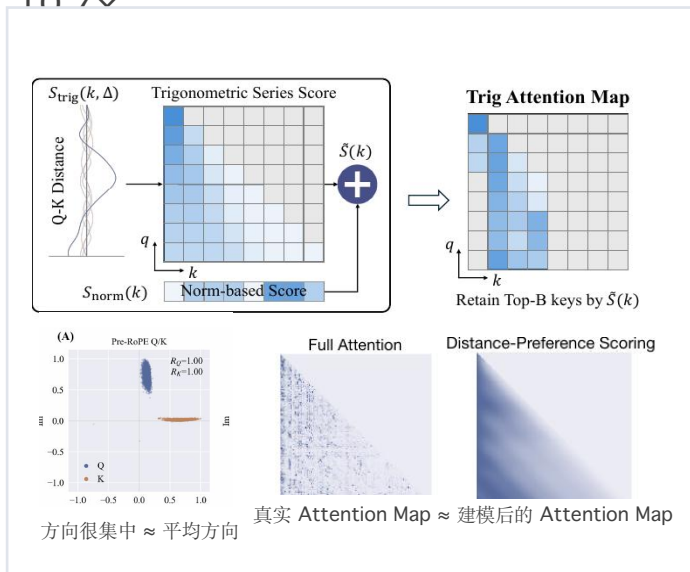


跨层合并与已有单层压缩方法对比

TriAttention

ICML 2026 · w/ MIT, NVIDIA

- pre-RoPE Q/K 的方向非常集中，可以用它们的平均方向近似表示
- 将平均方向带入 RoPE，得到只与相对位置有关的 Trigonometric Series Score；建模后的 Attention Map 与真实结构高度一致可以用来指导 kv 裁剪
- 在等精度下，吞吐提升 2.5x，KV Cache 压缩 10.7x



[1] He et al., ZipCache: Accurate and Efficient KV Cache Quantization with Salient Token Identification, NeurIPS 2024.

[2] Liu et al., MiniCache: KV Cache Compression in Depth Dimension for LLMs, NeurIPS 2024.

[3] Mao et al., TriAttention: Efficient Long Reasoning with Trigonometric KV Compression, ICML 2026.

■ 系统级 KV Cache 分层

▲ 热 · 快 · 小

GPU HBM

容量: 几十 GB · 带宽: ~TB/s

热 —— 正在生成的部分

CPU DRAM

容量: 几百 GB · 带宽: PCIe ~64 GB/s (约为 HBM 的 2%)

温 —— 刚被换出的部分

本地 SSD / NVMe

容量: 几十 TB · 带宽: ~GB/s

冷 —— DRAM装不下的溢出

远端 / 分布式存储

容量: 近乎无限 · 带宽: 受网络带宽限制 (RDMA)

跨实例 / 会话共享

▼ 冷 · 慢 · 大

在 Agent 场景中

热内存 (hot)

当前会话的近期轮次 / 工具输出 —— 常驻显存, 随时复用。

冷内存 (cold)

更早的轮次与长期事实 —— 推到 SSD / 远端, 只在需要时取回。

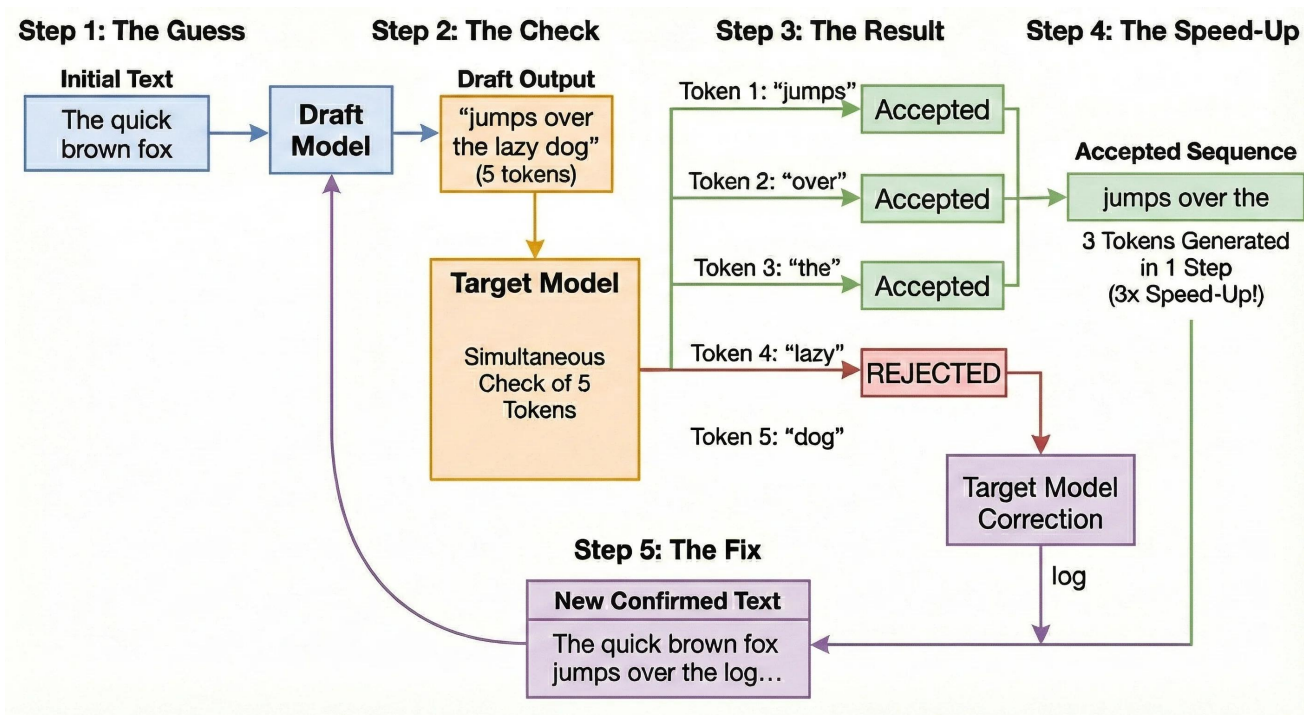
就像 OS 的存储层级: 推理系统把「热」的 KV 留在近处, 把「冷」的 KV 推到更便宜的层级。

[1] NVIDIA, Dynamo — low-latency distributed KV cache offload engine, 2026. [2] MinIO, KV Cache Offload as a Memory-Hierarchy Problem, 2026.



Efficient Decoding Paradigm

Speculative Decoding



研究动机:

- 低并发场景下, LLM 解码主要受内存带宽限制
- 推测解码旨在利用这部分闲置的计算能力

研究方法:

- 小型 Draft Model 快速串行/并行生成候选 Token
- 大型 Target Model 并行验证

主要特点:

- 无损
- 对并发水平高度敏感

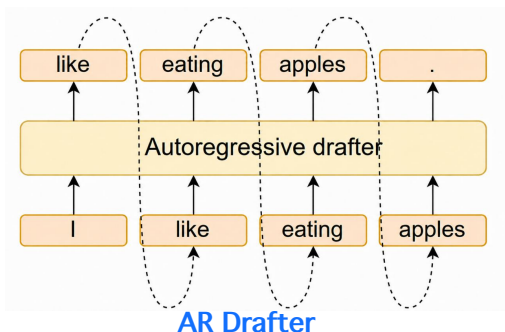
类别:

Training-free: LLaMA-70B Target + 8B Drafter

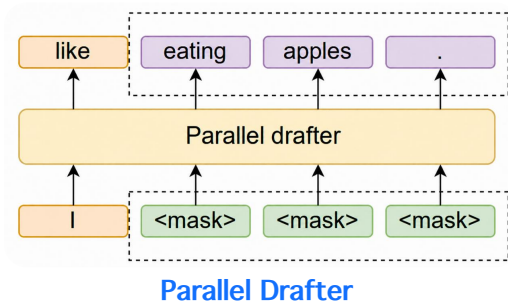
Training-based: EAGLE-3、DFlash、DSpark 等

目前, 基于训练的 **Drafter** 正在工业界被广泛采用, 例如 Qwen3.5 / 3.6 / 3.7 / 3.8 中的 **MTP**、DeepSeek 中的 **DSpark** 等。

自回归 Drafter vs 并行 Drafter



以 Eagle-3 为例，按顺序生成 draft Token



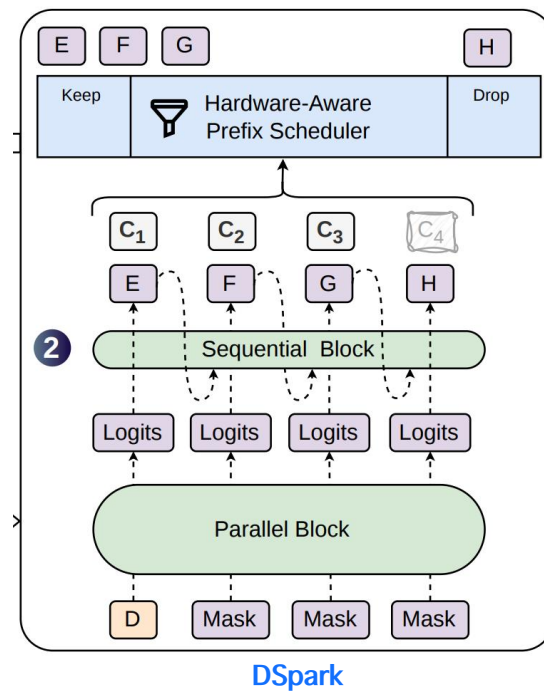
以 DFlash 为例，并行生成 draft Token

对于自回归 Drafter:

- 优势: **更好的对齐** — 目标模型是自回归模型
- 劣势: **draft 成本高** — 频繁调用 LM head, 限制 Drafter 规模

对于并行 Drafter:

- 优势: **draft 成本低** — Drafter 可扩展到更长的 acceptance length
- 劣势: **对齐偏差** — draft token 之间不存在依赖关系;
验证成本更高 — 需要验证更多 Token; 若 acceptance length 较短则加速收益有限



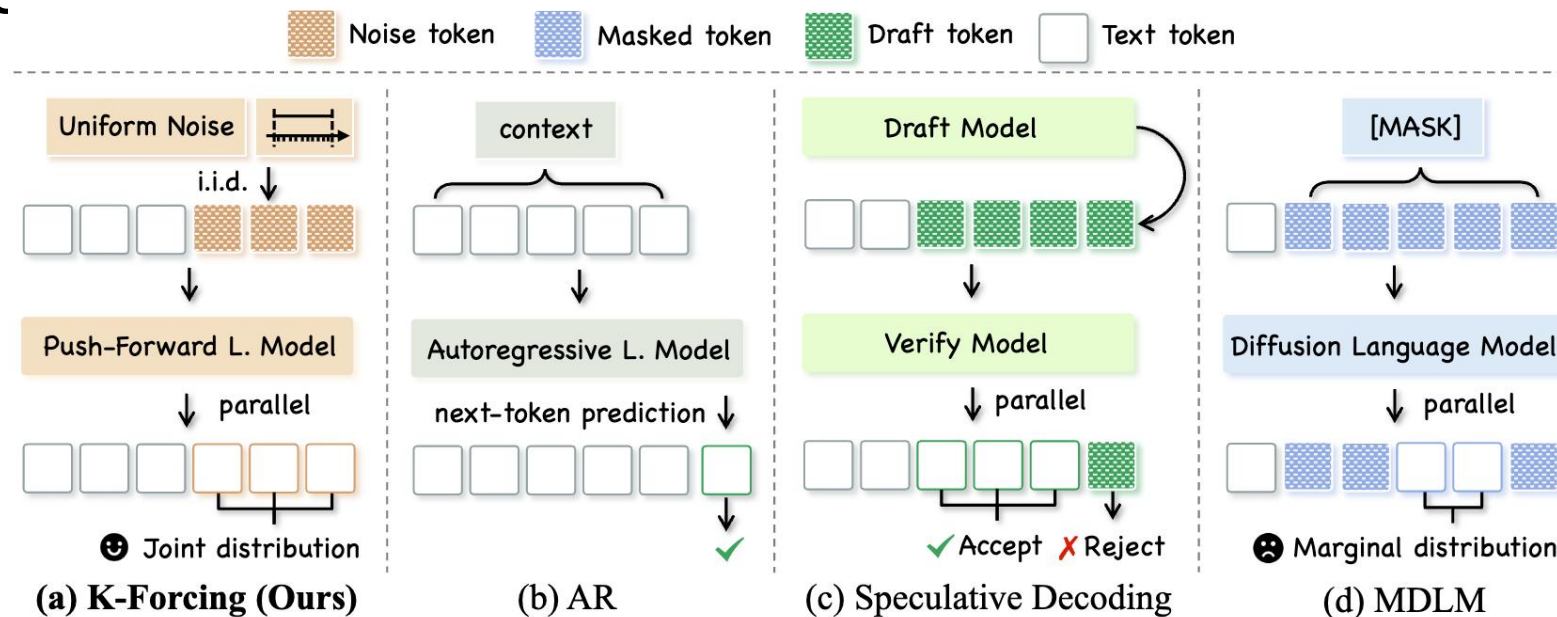
并行生成 draft Token
按顺序修正 logits

DSpark 结合自回归 Drafter 与并行 Drafter 的优势: 在算力打满的情况下增加 acceptance length

- 对齐偏差: 轻量模块**按顺序修正并行 logits**
- 验证成本: 仅验证由 conf head 评分的**高置信度 draft token**



K-Forcing: Joint Next-K-Token Decoding via Push-Forward Language Modeling



当前范式的不足:

AR: 一步一个 token, 解码受显存带宽限制

Speculative decoding: 接受长度可变, 但是batch 难对齐。

MDLM: 并行预测边缘分布, 难保持联合依赖 (the joint conditional distribution over the next k tokens)

解决方案:

K-Forcing: 用随机噪声映射未来 K 个 token。固定长度输出 + 一次 causal forward, 无需额外 verify。

K-Forcing: Joint Next-K-Token Decoding via Push-Forward Language Modeling

动机:

把 AR 模型的能力迁移到一次生成多个 Token 的模型上, 从而提升生成效率。

基线:

教师在真实数据上前向一遍, 于每个词元的 CDF 区间内采 $z^* \sim U(l_t, u_t)$ 即可廉价获得 (z^*, x) 对; 但该噪声反演在区间过窄时会翻转 token, 不够稳健。

训练:

阶段 1 | AR $\rightarrow$ PFLM($k=1$)

直接学习前向映射: 先抽 $z \sim U(0,1)$, 再由教师用逆 CDF 生成 token, 无需噪声反演, 更稳健。

阶段 2 | Self-Forcing 蒸馏

教师先生成 k 个 token, 学生一次生成 $2k$ 个。

结果:

窗口逐步扩大: $1 \rightarrow 2 \rightarrow 4$

$k=2$: 约 1.6 $\times$, 质量接近 AR

$k=3$: 约 2.4 $\times$, 质量略有下降

$k=4$: 约 2.4–3.5 $\times$, 质量下降更明显; 固定 k 让批处理保持对齐。

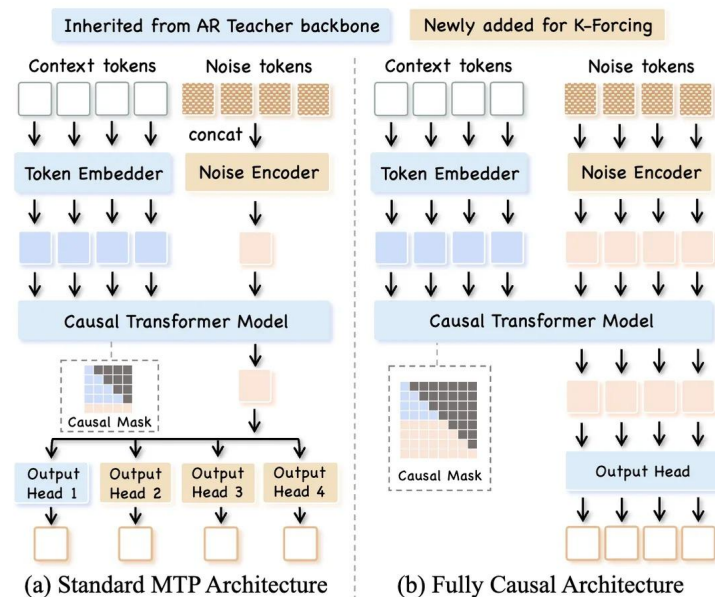


Figure 2 (a) Standard MTP: k noise variables are concatenated into one token; k independent heads decode future tokens from its hidden state. (b) Fully causal: each noise variable forms a separate token under causal attention and is decoded by a shared head. Both reuse the AR backbone with a sinusoidal+MLP noise encoder.

Dataset	Method	Quality		Throughput (k tokens/s, speedup)		
		Gen-PPL↓	Win vs. AR↑	BS=4	BS=16	BS=128
LM1B	AR	104.8	—	0.54 (1.00 $\times$)	2.03 (1.00 $\times$)	15.4 (1.00 $\times$)
	K-Forcing ($k=2$)	107.2	50.2%	0.84 (1.56 $\times$)	3.48 (1.71 $\times$)	24.7 (1.60 $\times$)
	K-Forcing ($k=3$)	117.1	45.1%	1.27 (2.35 $\times$)	4.99 (2.46 $\times$)	35.6 (2.31 $\times$)
	K-Forcing ($k=4$)	127.6	42.9%	1.69 (3.13 $\times$)	6.77 (3.33 $\times$)	46.5 (3.02 $\times$)
OWT	AR	42.64	—	0.53 (1.00 $\times$)	1.99 (1.00 $\times$)	9.61 (1.00 $\times$)
	K-Forcing ($k=2$)	32.82	46.9%	0.85 (1.60 $\times$)	3.36 (1.69 $\times$)	11.9 (1.24 $\times$)
	K-Forcing ($k=3$)	29.67	42.8%	1.28 (2.42 $\times$)	5.03 (2.53 $\times$)	17.5 (1.82 $\times$)
	K-Forcing ($k=4$)	24.97	39.4%	1.70 (3.21 $\times$)	6.91 (3.47 $\times$)	22.9 (2.38 $\times$)

LLM推理框架

框架	主要定位	关键技术	适用场景
SGLang	生产级推理服务	RadixAttention; prefix cache; structured outputs; spec decode	低延迟 / 高吞吐 API 与集群
vLLM	通用推理服务	PagedAttention; continuous batching; prefix cache; speculative decoding	模型与硬件生态广泛
FlashInfer	GPU kernel 库	Attention、paged KV、GEMM、量化与 MoE kernel	自建推理栈与 kernel 后端
KTransformers	CPU-GPU 混合 MoE	CPU expert kernel; offload / 调度; 量化	显存受限下运行大型稀疏模型
TensorRT-LLM	NVIDIA 推理运行时	In-flight batching; paged KV; quantization; multi-GPU; spec decode	面向 NVIDIA 优化的生产部署

Part 2: Efficient Multimodal Inference

第二层： 应用层 – 多模态理解和生成

长视频的开销有多大？

长视频理解

Qwen2.5-VL-7B · FP16 · 1 fps · 448×448 · 1×H100

	30 分钟	2 小时
视觉 Token 数	230 K	920 K
KV Cache (FP16)	≈ 12 GB	≈ 49 GB
Prefill 时间	≈ 1 分钟	≈ 15 分钟
Decode 速度	≈ 120 tok/s	≈ 50 tok/s

长视频生成

Wan2.1-14B · 720p · 16 fps · 50 steps · FP16 · 1×H100

	5 秒	60 秒
Latent Token 数	75.6 K	868 K
KV Cache (AR/流式)	≈ 62 GB	≈ 710 GB
计算量	3.4×10^{17}	3.2×10^{19}
总耗时	≈ 9 分钟	≈ 15 小时

Attention 主导: 在 230K / 920K token 下，全序列 attention 分别占 87% / 96% 的 FLOPs；自回归 / 流式生成设定下，仅 1 分钟 720p 视频的 KV Cache 就已超过单张 80GB GPU。

内存 (KV Cache) 与计算量 ($O(N^2)$ Attention) 同步增长



Efficient Long Visual Understanding

范式一：压缩输入序列

LongVLM: Efficient Long Video Understanding via Large Language Models

Yuetian Weng, Mingfei Han, Haoyu He, Xiaojun Chang, Bohan Zhuang (ECCV 2024 oral)

Motivation

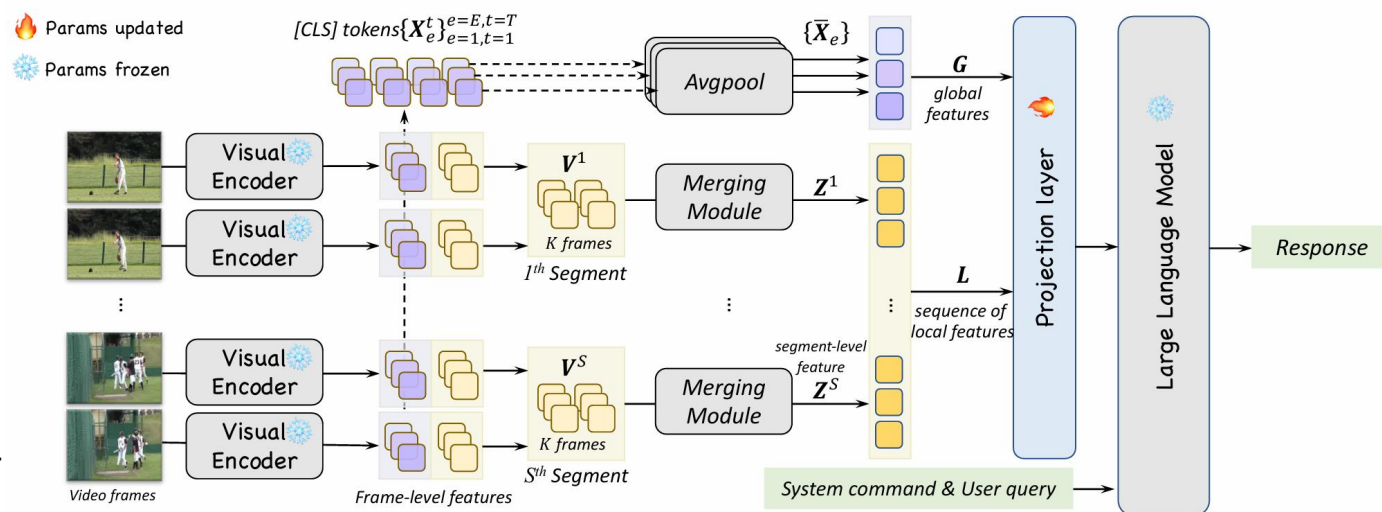
- 之前方法直接压缩视频 token，易丢局部事件细节

Method

- LongVLM: 长视频切成短片段，段内用层次化 token merging 压缩局部特征，再把全局 [CLS] 语义前置 → LLM 同时看到全局上下文与时序局部事件

Result

- 100 帧视频压到 305 token，仍保留细节信息
- 视频理解平均分 2.42 → 2.89，优于 Video-ChatGPT
- 零样本问答：三个基准均超此前最佳 1.9–2.8 个百分点



在 LLM 看到输入前先把序列压短 → 每层、每步生成的计算成本同步下降

■ 范式二: Dynamic Sparse Attention

Motivation

- Attention map 高度稀疏, 但稀疏模式随任务与层大幅变化 — 固定稀疏比既损精度、也难达最优加速

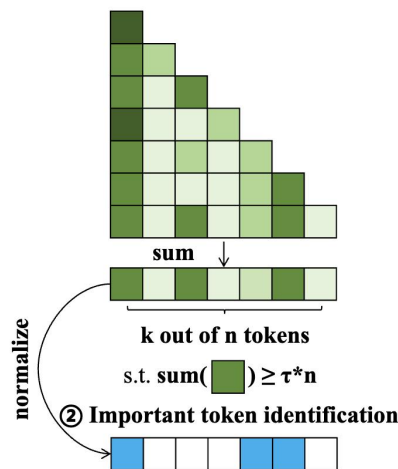
Method

- ZipVL (ICCV 2025): 逐层动态估计 sparsity pattern (probe-token scores) → 省 prefill 计算、省 decode 阶段 KV Cache I/O
- Sparsity Forcing (ICLR 2026): 用 GRPO 让模型适应稀疏注意力, 引入 sparsity reward → 精度不降拉高稀疏度

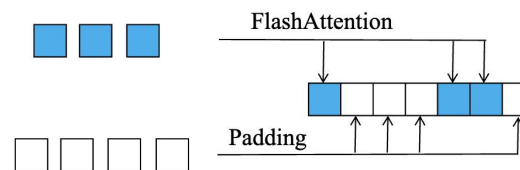
Results

- Qwen2-VL-7B: 保留 23.8% token, video 62.1 → 61.9, image 70.9 → 70.8
- Qwen2.5-VL-7B: 保留 25.9% token, video 65.2 → 64.9

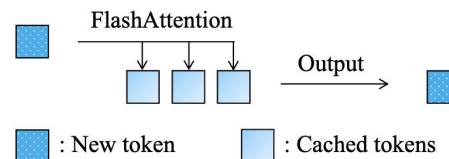
① Dynamic Ratio of Important Tokens



③ Token-level Sparse Attention (Prefill Phase)



④ Token-level Sparse Attention (Decoding Phase)



ZipVL framework: dynamic ratio → token-level sparse attention

保留完整序列, 仅让重要 token 参与注意力计算

[1] Y. He, F. Chen, J. Liu, et al. ZipVL: Efficient Large Vision-Language Models with Dynamic Token Sparsification. [ICCV 2025](#).

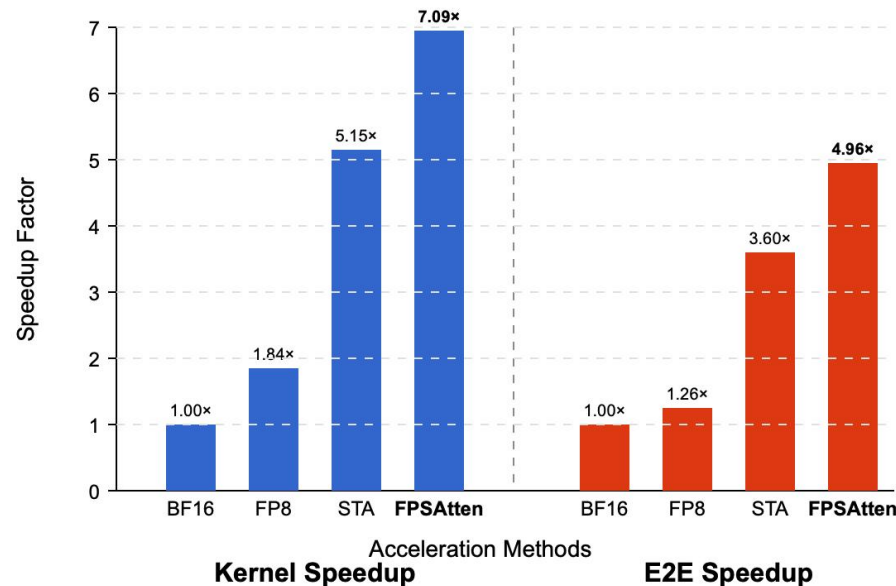
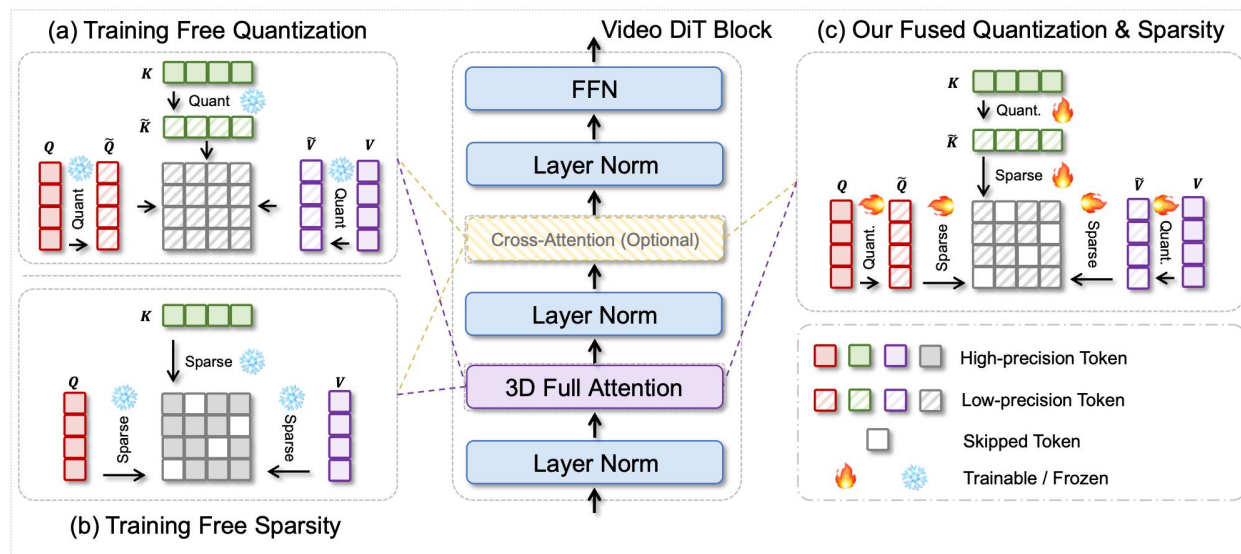
[2] F. Chen, Y. He, L. Lin, et al. Sparsity Forcing: Reinforcing Token Sparsity in MLLMs. [ICLR 2026](#).



Efficient Long Visual Generation

Sparse Attention & Quantization

FPSAttention: Training-Aware FP8 and Sparsity Co-Design for Fast Video Diffusion (NeurIPS 2025 [Spotlight](#))



Wan2.1-14B, 720P

1. 统一的 3D Tile-wise 粒度，用于联合 FP8 量化与稀疏注意力
2. 可随Noise Schedule自适应调整的粒度
3. 原生、硬件友好的 Kernel，并利用 FlashAttention 进行加速

Sparse Attention & Step Distillation

BLADE: Block-Sparse Attention Meets Step Distillation for Efficient Video Generation

Youping Gu, Xiaolong Li, Yuhao Hu, Minqi Chen, Bohan Zhuang (ICLR 2026)

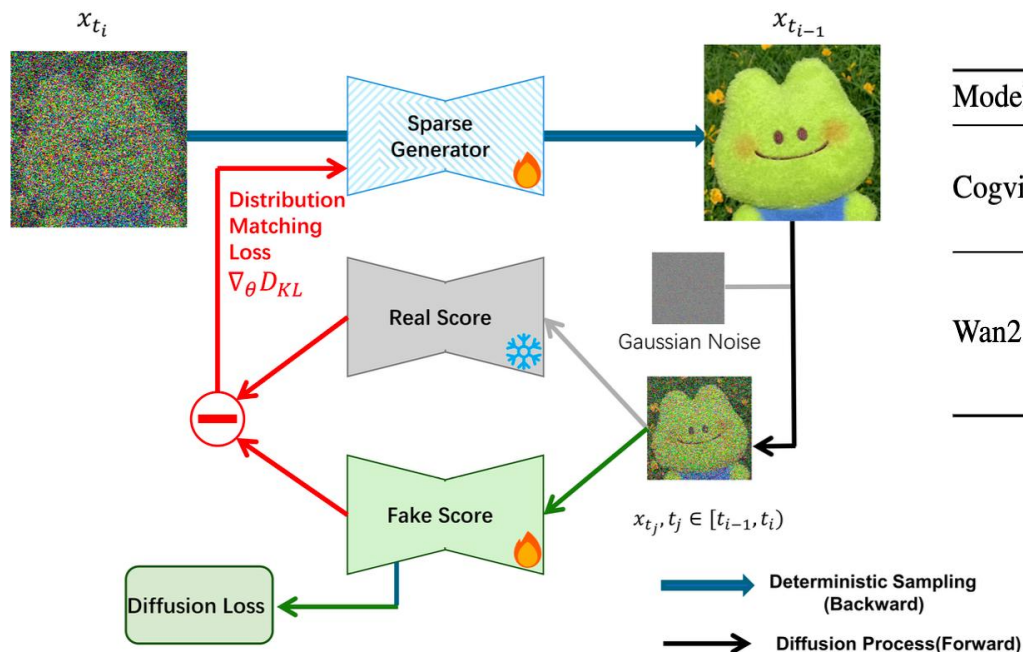


Table 1: Video Quality Evaluation on VBench-2.0.

Model	Method	Sparsity	Total	Creativity	Commonsense	Controllability	Human	Physics	Speedup
CogvideoX-5B	Baseline	-	0.534	0.458	0.523	0.341	0.808	0.539	1×
	FA2	-	0.539	0.458	0.498	0.354	0.813	0.570	7.93×
	ASA_G	0.82	0.569	0.546	0.514	0.367	0.802	0.618	8.89×
Wan2.1-1.3B	Baseline	-	0.563	0.508	0.549	0.338	0.820	0.600	1×
	FA2	-	0.580	0.631	0.485	0.311	0.841	0.631	9.37×
	STA	0.74	0.528	0.504	0.471	0.265	0.855	0.543	10.53×
	ASA_G	0.8	0.570	0.472	0.532	0.312	0.918	0.617	14.10×

50步基线 -> 8步蒸馏

Wan2.1-1.3B: **14.10×** 端到端加速

CogVideoX-5B: **8.89×**

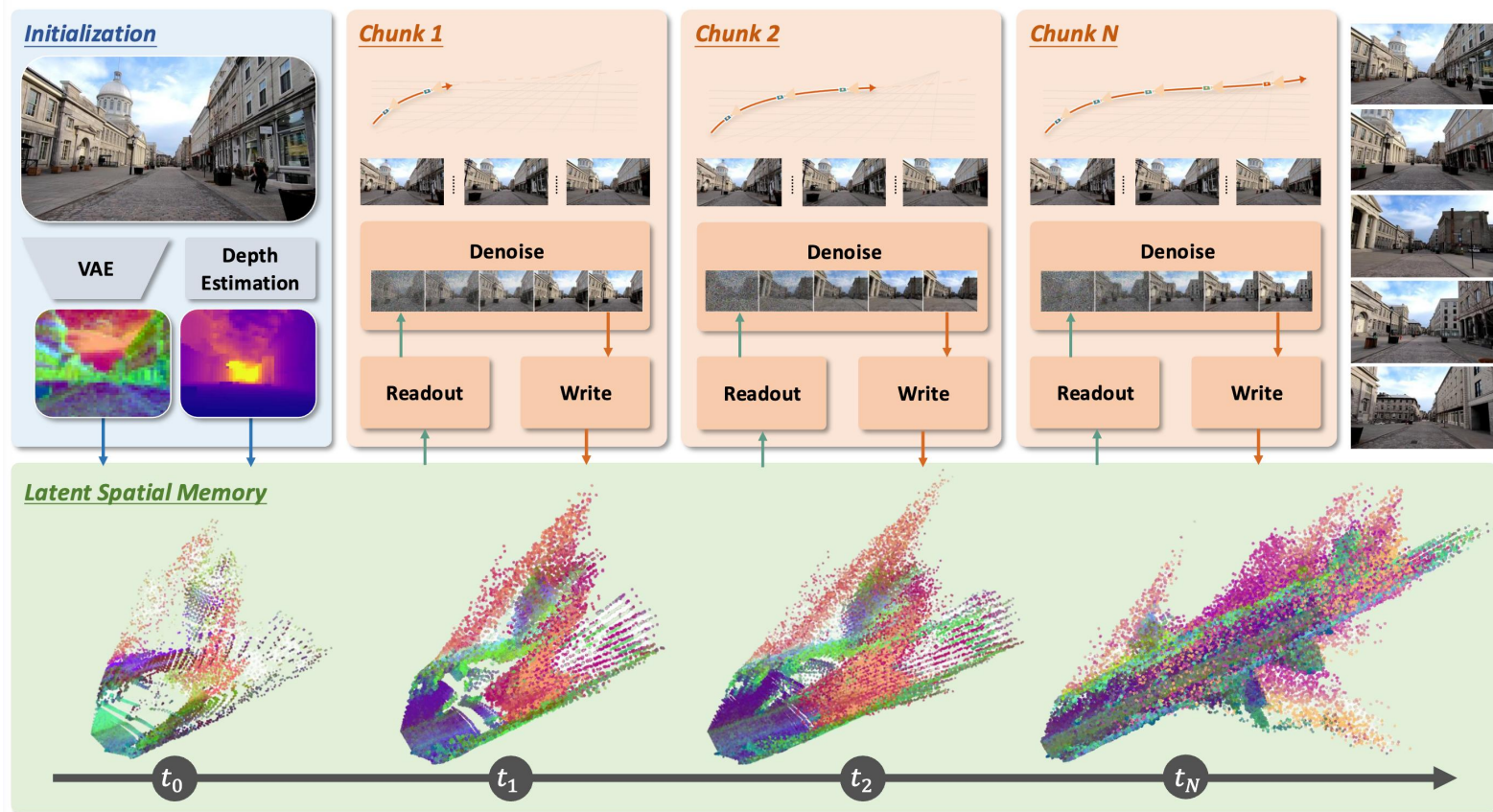
Vbench-2.0 CogVideoX score: **0.569**

动机: 融合步数蒸馏(DMD)和动态的块稀疏注意力

■ 面向 Video World Model 的 Latent Spatial Memory

Latent Spatial Memory for Video World Models

Weijie Wang, Haoyu Zhao, Yifan Yang, Feng Chen, Zeyu Zhang, Yefei He, Zicheng Duan, Donny Y. Chen, Yuqing Yang, Bohan Zhuang ([arXiv 2026](#))



视频世界模型的瓶颈:

- 为了保持场景一致性, 维护显式3D记忆开销巨大, 占用整个生成管线**超过60%**的时间
- 显式3D记忆是视频世界模型的重要性能瓶颈

Method & Results:

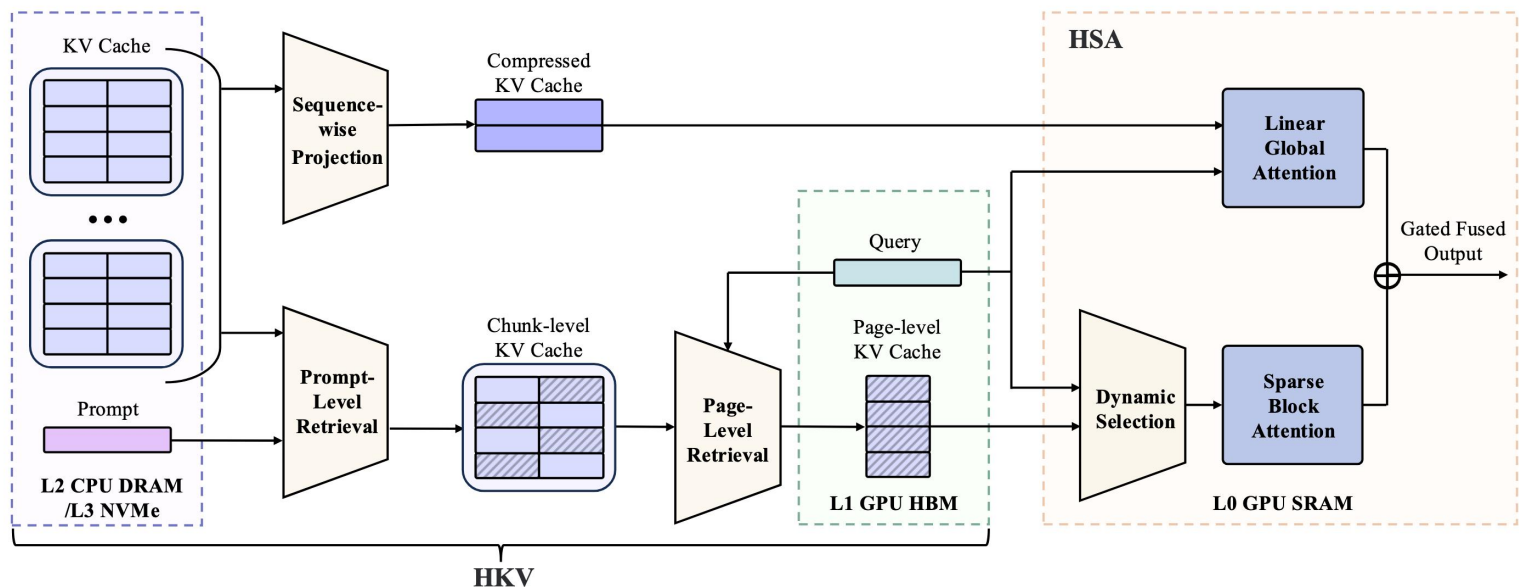
- 将显式3D记忆放到 Diffusion 隐空间中进行维护, 极大降低维护3D记忆的成本
- 视频生成最高加速**10.57倍**
- 3D记忆现存最高降低**55倍**

WorldAttention: 缓存与计算的系统协同设计

WorldAttention: An Efficient Attention Architecture for Interactive Video World Models

Zeyu Zhang, Jinyuan Mao, Dakai An, Wangbo Zhao, Hanfeng Lu, Jiasheng Tang, Yinghao Yu,

Wei Wang, Bohan Zhuang



当前长视频生成挑战:

- 滑动窗口会遗忘历史, 完整 KV Cache 会占满显存
- 缺乏成熟的缓存管理策略, 粗粒度检索造成冗余
- KV 重排与数据搬运成为主要延迟来源

WorldAttention:

- KV 分层缓存: GPU HBM、CPU DRAM、NVMe
- 分页存储、多级检索: 先选 chunk, 再按需加载 page
- HSA 混合注意力: 线性全局分支 + 稀疏细节分支

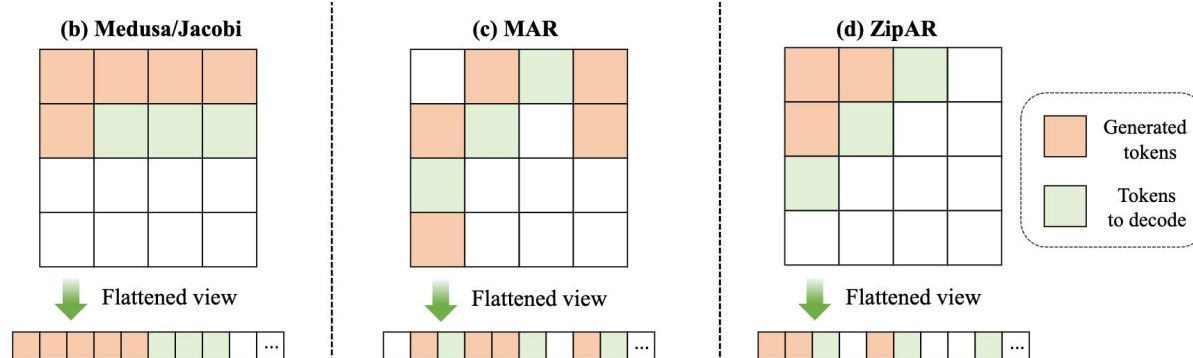
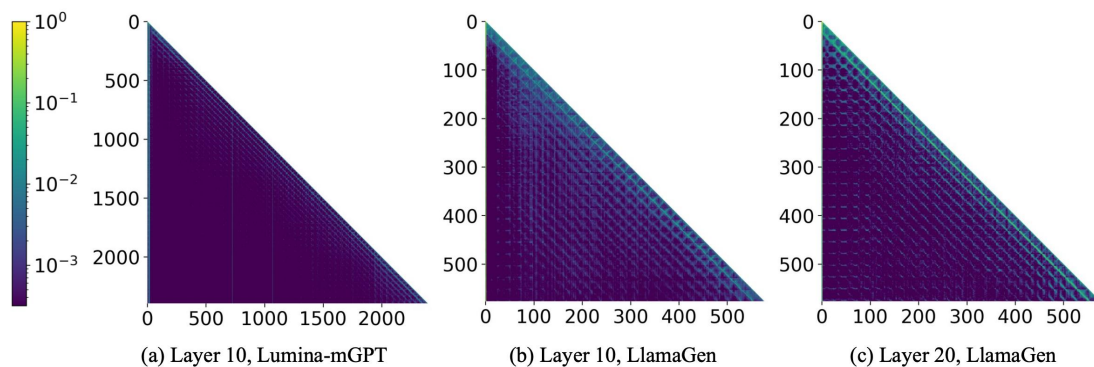
在保持长时一致性的同时, Kernel 加速最高 **14.02x**; 32帧重缓存下达到 **22.0 FPS (vs. 7.1 FPS)** ; 端到端提升

2.21x

■ 面向自回归 (AR) 视觉生成的并行解码

ZipAR: Parallel Autoregressive Image Generation through Spatial Locality

Yefei He, Feng Chen, Yuanyu He, Shaoxuan He, Hong Zhou, Kaipeng Zhang, Bohan Zhuang (ICML 2025)



Lumina-mGPT / LlamaGen 的注意力图：质量集中在对角线周围的狭窄带内。

范式对比图

Motivation Raster-AR 模型生成能力强，但逐 token 串行解码效率低，如何既保持生成能力又高度并行生成？

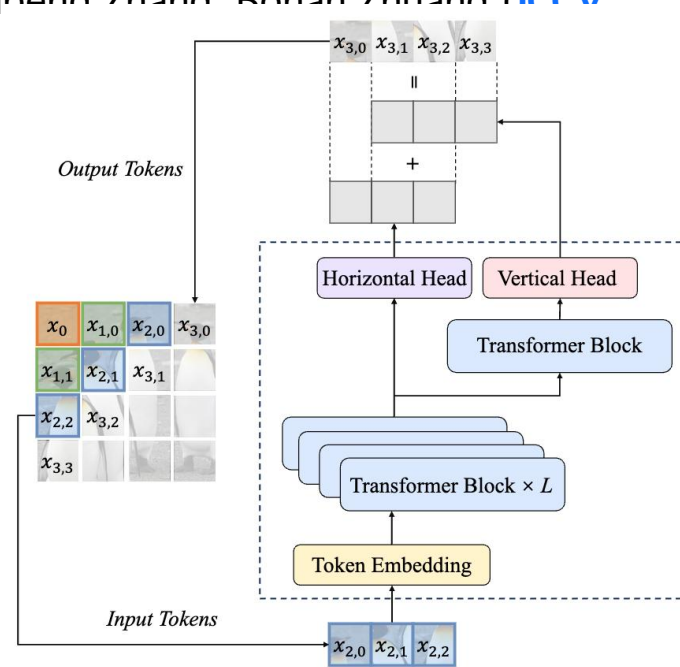
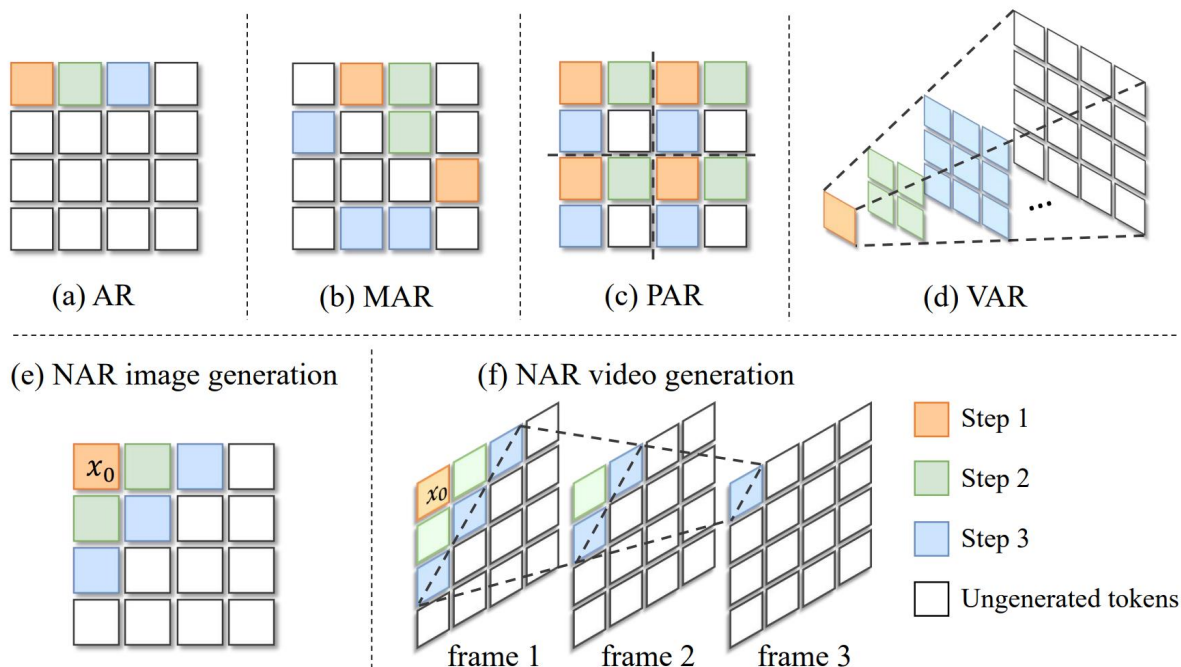
Method 解码相邻行的局部窗口，而不是逐行完成；窗口大小随行调整

Results LlamaGen-XL: 1024 $\rightarrow$ **184 steps** and 33.17 s $\rightarrow$ **5.51 s (-83%)**
CLIP 0.287 $\rightarrow$ 0.280; 无损设置下 (ZipAR-16) 将延迟降低了 **47%**

Neighboring Autoregressive Modeling for Efficient Visual

Generation

Yefei He, Yuxuan He, Shengxin He, Fan Yang, Chen Liang, Zhou, Kaipeng Zhang, Bohan Zhuang (ICCV)



混合logits采样: 避免边缘分布

Method

保留 next-token 训练目标, 增加水平与垂直预测头, 沿反对角线并行解码多个 token

邻域解码将 $O(n)$ 的顺序步骤简化为 $O(\sqrt{n})$ 规模的并行, 且质量保持不变。

FlashAR: Efficient Post-Training Acceleration for Autoregressive Image Generation

Generation

Junkang Zhou, Yefei He, Feng Chen, Weijie Wang, Bohan Zhuang ([arXiv 2026](#))

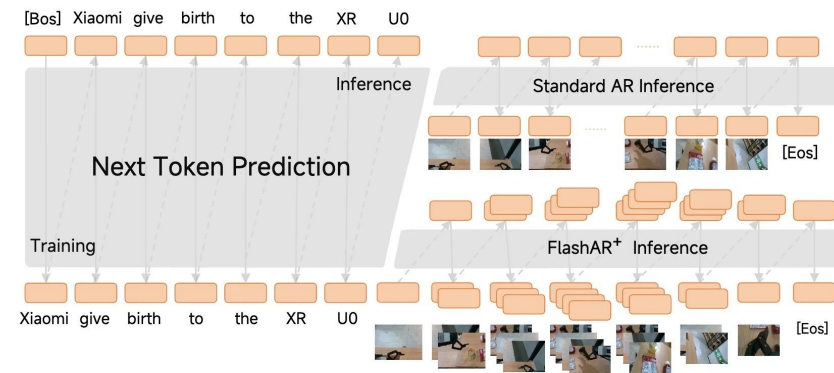
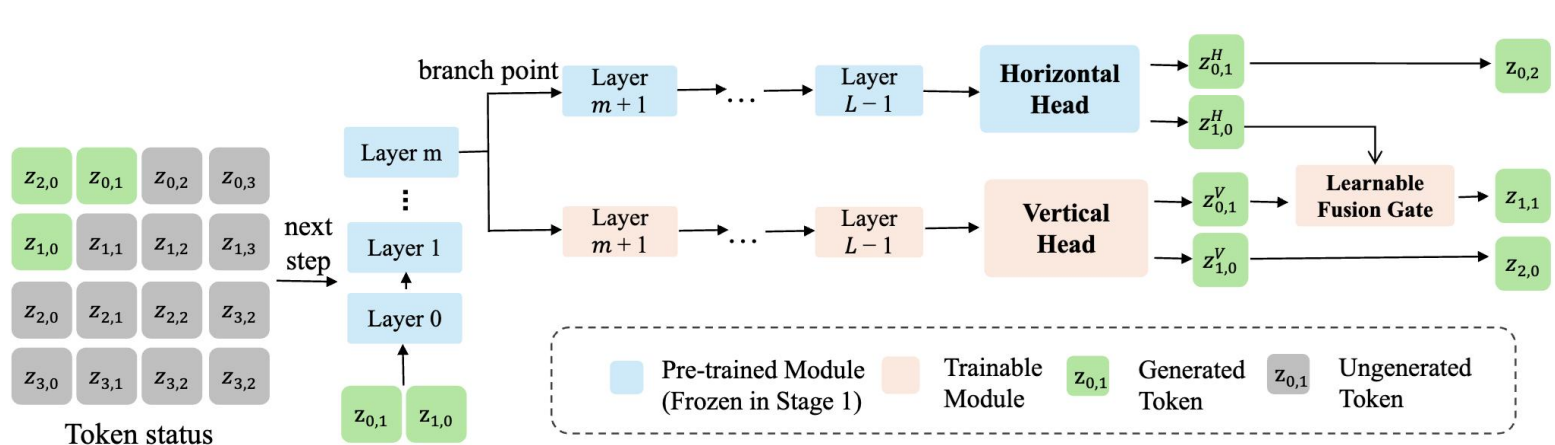


Figure 3 Overview structure of Xiaomi-Robotics-U0. The model is trained end-to-end at scale with a unified next-token prediction objective. An efficient inference extension is provided along.

Results

Across model scales: **throughput $\uparrow$ 4.8 $\times$**

Emu3.5-Image: **130.10 s $\rightarrow$ 5.68 s (22.9 $\times$) with only 1% data**

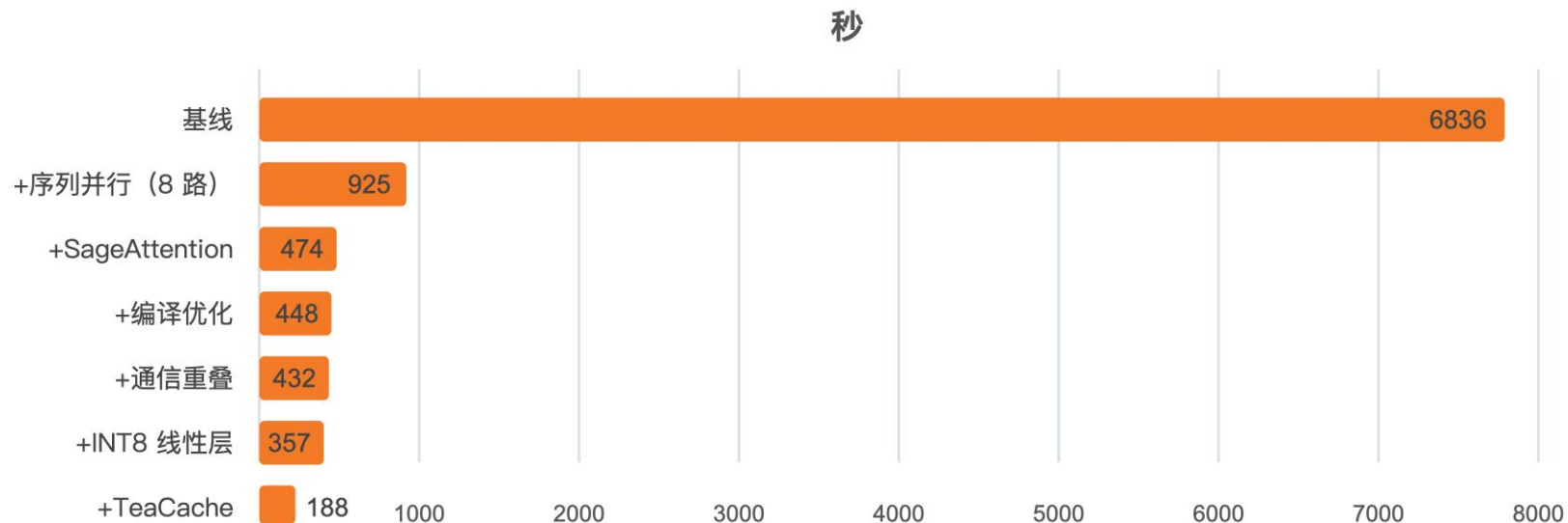
Xiaomi-Robotics-U0: **实现近 83 $\times$ acceleration, efficient enough for practical use**

[1] Xiaomi-Robotics-U0: Unified Embodied Synthesis with World Foundation Model, arXiv:2607.11643, 2026.

VideoGen 推理基础设施 – DAX

DAX: 让 Wan2.1 视频生成推理加速 36 倍 (without step distillation)

测试条件: Wan2.1 T2V 14B, 720p, 生成 5 秒视频, 50 步采样; 单卡 NVIDIA H20。单位: 秒/视频。



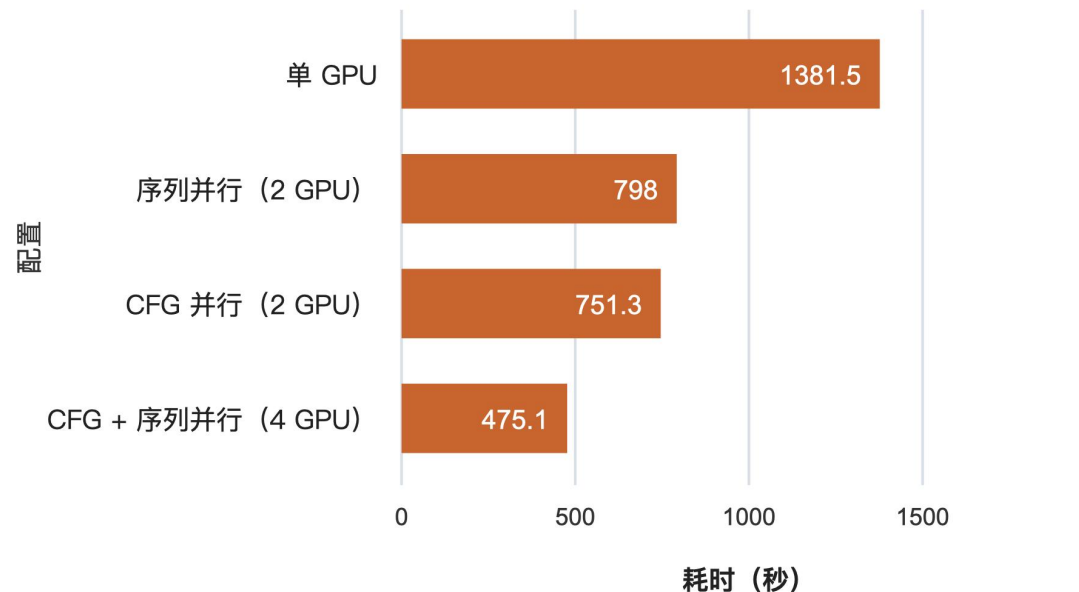
- 量化: FP8/INT8 GEMM 与 SageAttention2 加速注意力计算。
- 序列并行: 通过调优通信与计算重叠, 提升并行效率。
- TeaCache: 跳过不重要的去噪步骤, 加速 DiT 推理。
- Torch.compile: 融合量化与通信算子, 减少额外开销。

<https://github.com/RiseAI-Sys/DAX> 2025.07

VideoGen 推理基础设施 – SGLang Diffusion

SGLang-Diffusion: 面向图像与视频生成的高性能推理与服务框架

不同并行配置下的 SGLang Diffusion 性能

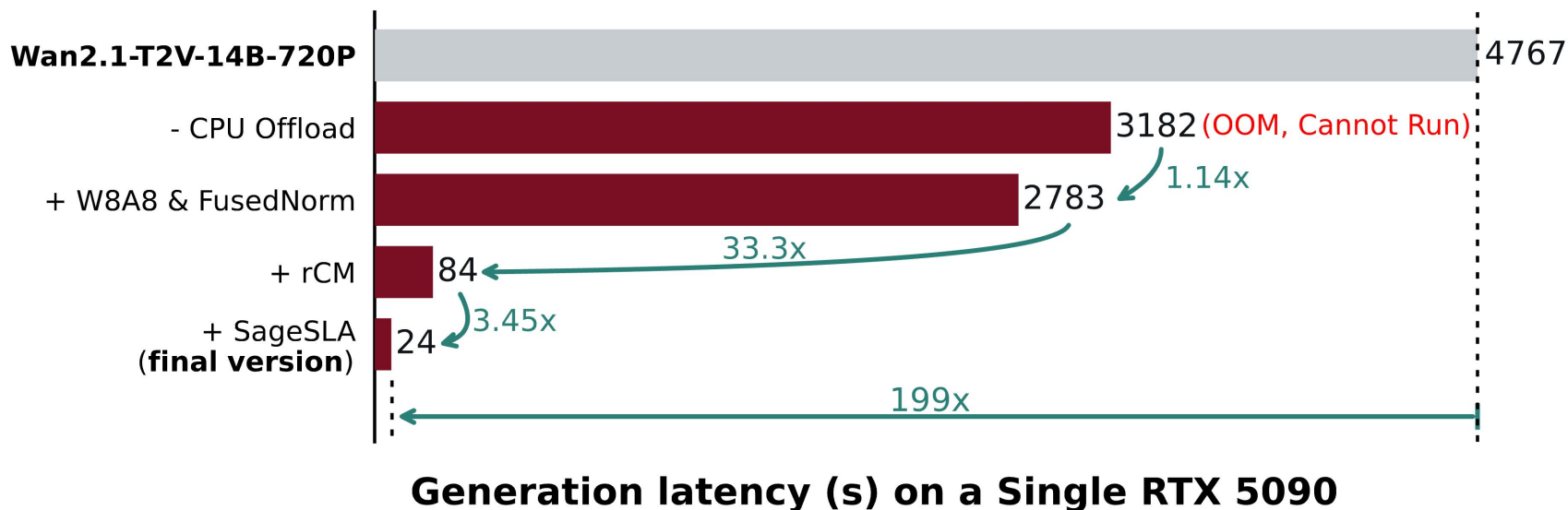


- 注意力后端: FlashAttention、SageAttention 及多种稀疏注意力实现。
- 量化与缓存: FP8/NVFP4/INT4 量化, 以及 Cache-DiT 和 TeaCache。
- 并行服务: Ulysses/Ring 序列并行, 以及 TP/FSDP/CFG 并行。

[SGLang Diffusion](#)

VideoGen 推理基础设施 – Turbo Diffusion

TurboDiffusion: 模型与系统协同设计的视频扩散加速框架 (加速 100–200 倍)



- **W8A8 量化:** 将权重和激活值量化, 加速线性层并降低显存。
- **rCM 步数蒸馏:** 将扩散采样减少到 1–4 个去噪步骤。
- **SageSLA:** 结合低比特 SageAttention 与可训练稀疏线性注意力。
- **算子优化:** 融合归一化与量化 GEMM 算子。

<https://github.com/thu-ml/TurboDiffusion>

Inferix: 面向 AR-Diffusion 世界模型的推理引擎

Efficient inference stack

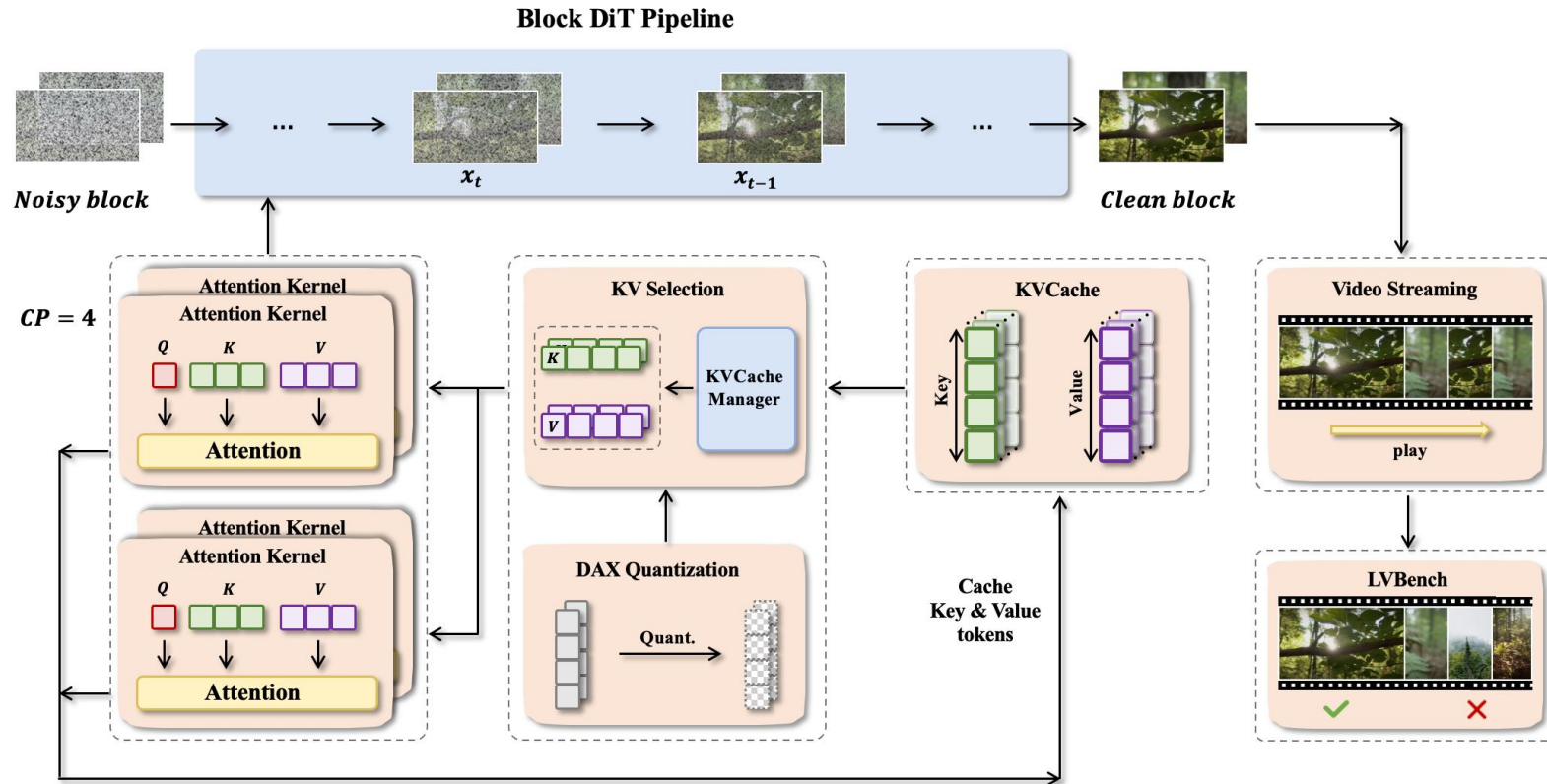
- Tiered KV cache management
- INT8/FP8 quantization
- CP/TP parallelism

Interactive world generation

- Continuous prompts
- pause/resume
- live prompt switching
- RTMP/WebRTC streaming support

Deployment-ready

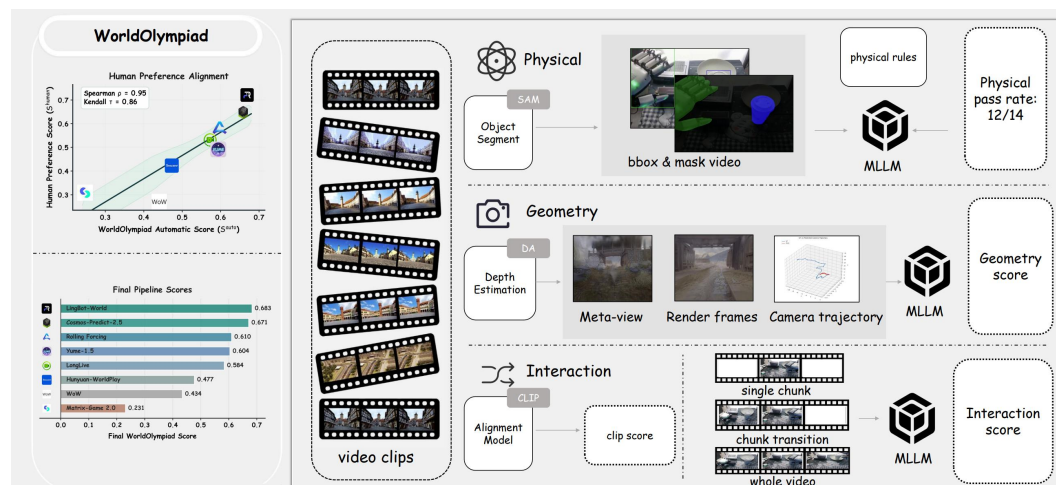
- One API to deploy easily
- built-in profiling
- runs on 16 GB consumer GPUs



Block-DiT pipeline: KV selection & cache manager, DAX quantization, streaming output.

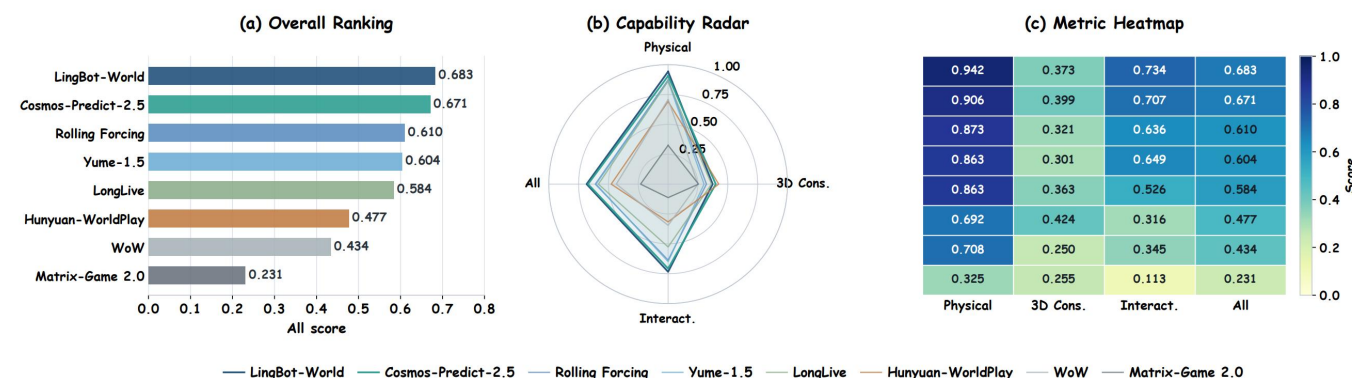
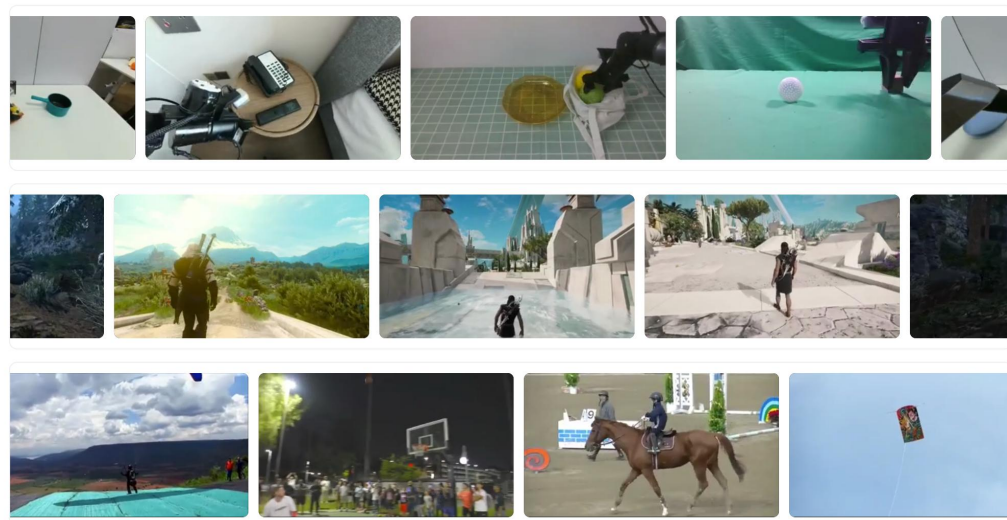
Inferix serves block-diffusion world models interactively, in real time — even on consumer GPUs

World Model Benchmark: WorldOlympiad



世界模型原生的测评基准:

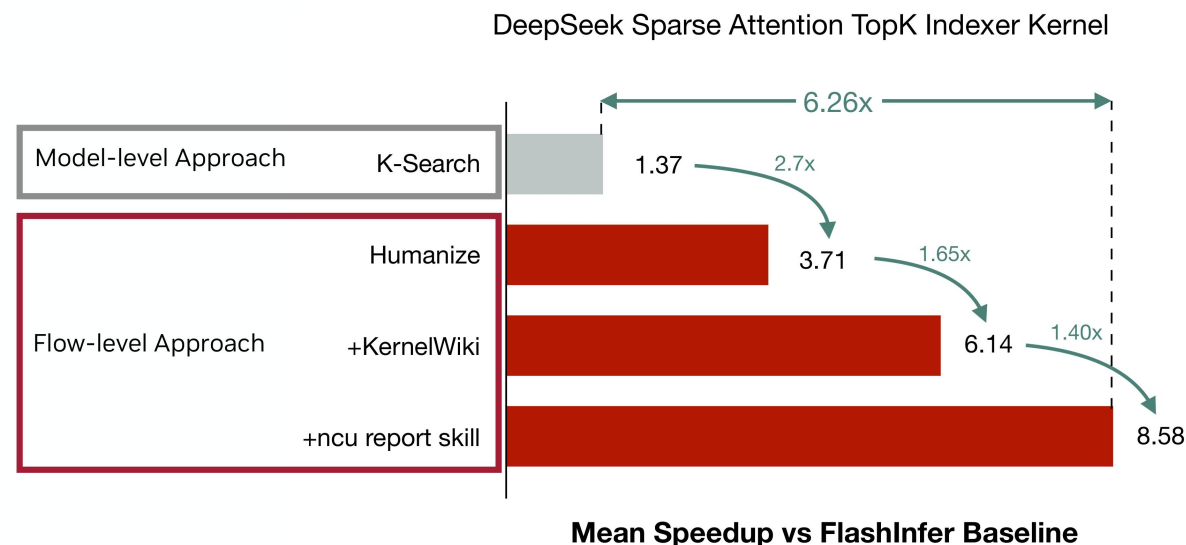
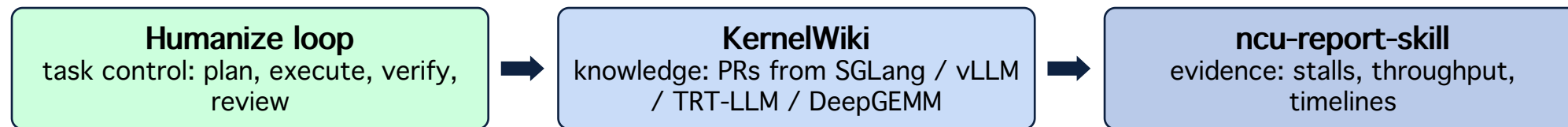
- 1000条高质量的长视频数据
- Physical+Geometry+Interaction, 覆盖世界模型一致性和可交互性的指标
- 现实场景+游戏模拟+具身操作, 覆盖世界模型最重要的下游任务



Part 2: Efficient Multimodal Inference

第三层： 进化层 Recursive Self-improvement
(RSI)

■ 面向 Kernel、Infra 与硬件优化的 Agent Loop



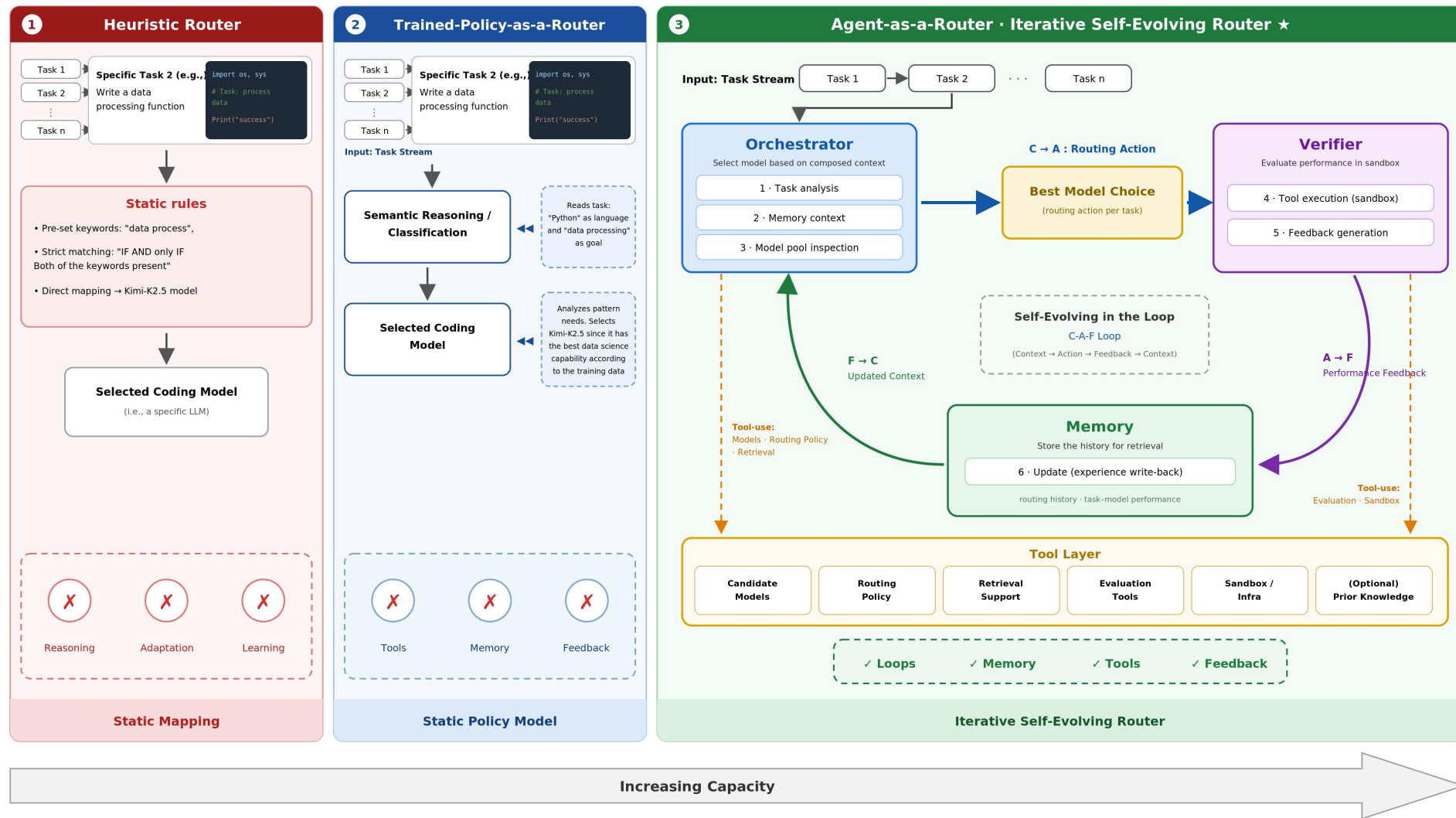
Over 200+ teams:

- #1 MoE track — Full-Agent
- #2 DSA track — Full-Agent
- #3 GDN track — Full-Agent

- No single trick — workflow, knowledge, and profiler evidence compound to 6.26x.
- Implement: CC, Review: Codex
- DSA indexer: 1.37x → 19.08x
- DSA attention: 6.15x → 15.22x
- FP8 MoE: 0.27x → 1.31x (post-contest, vs closed-source TRT-LLM baseline)
- Not magic everywhere: GDN decode 0.92x — verifier-grounded honesty

[1] L. Zhu, Advancing Token Productivity with Agent Loops, NVIDIA Research, 2026. [2] Ligeng Zhu et al., KDA: Kernel Develop Agents for Blackwell, NVIDIA Research, 2026 (figure adapted). [3] github.com/mit-han-lab/kernel-design-agents.

Agent-as-A-Router



在 3,000 个编程任务上, Agentic Router 以不到 Opus 一半的成本取得了更高的平均得分。

分布内测试 (n=2,919)

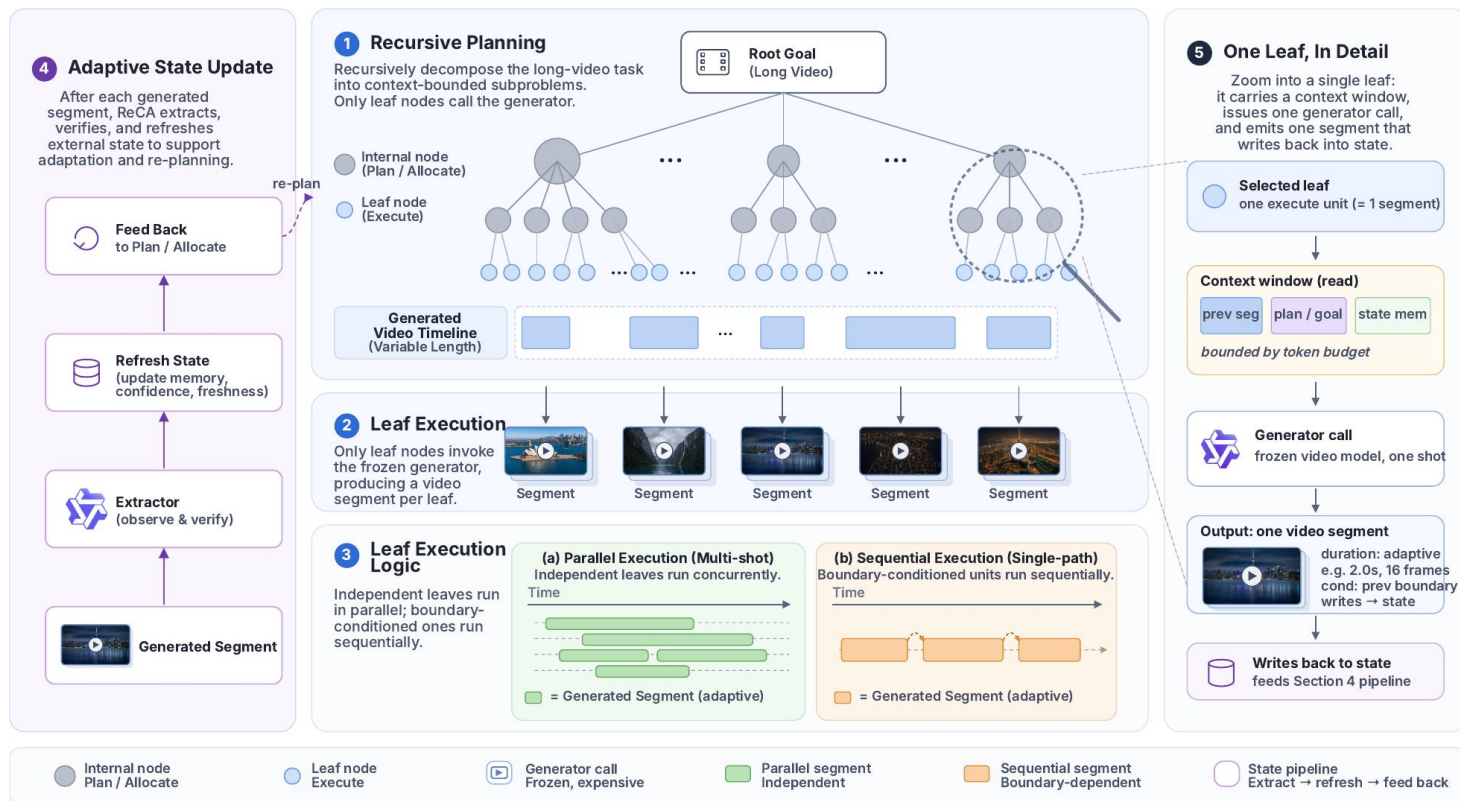
- 平均得分 **49.98%** vs Opus **43.83%**
- 总成本 **\$13.21** vs **\$34.02** (约 39%)
- 性价比 3.79 vs 1.29

真实 OOD 任务 (n=176)

- 平均得分 **62.50%** vs Opus **57.14%**
- 总成本 **\$52.97** vs **\$89.28**

Zhou et al. Agent-as-a-Router: Agentic Model Routing for Coding Tasks. arXiv:2606.22902, 2026.

Agentic Video Generation: ReCA



ReCA形成的背景：目前视频生成的问题

- 单镜长镜头 (LongLive 等)：能延长连续画面，但缺少多镜叙事
- 多镜短镜头 (MovieAgent 等)：能组织多个镜头，但单镜通常较短且简单拼接
- 电影叙事需要：多镜头协同，长短镜头交错且跨镜连续

ReCA系统层

多镜规划：Plan 递归拆成可变时长 shots

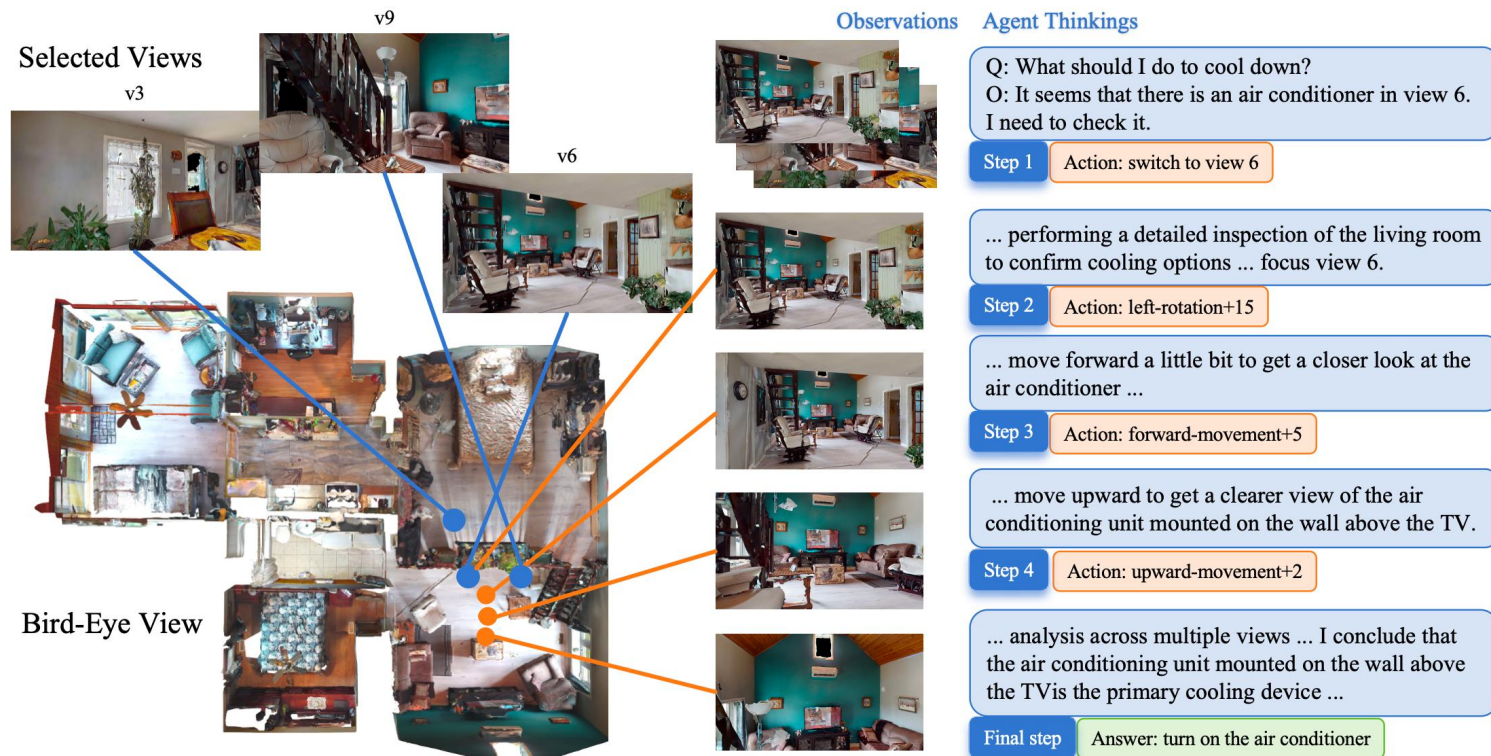
跨镜连续：镜头间的状态隔离 (Allocate)，逐段回写更新状态 (Refresh)

组织编排：独立 leaves 并行，依赖 leaves 串行

ReCA 无需重训视频生成器，实现跨镜头连续的分钟级电影叙事，在 ViStory、VBench 等评测上的综合得分提升 8-16%，单位视频生成开销降低 18-22%。

A. Liu, J. Xing, C. Mao, et al. ReCA: Multi-Shot Long Video Extrapolation via Recursive Context Allocation. arXiv:2605.26525, 2026.

Agent 空间推理框架: Chain-of-View Prompting (CoV)



Motivation:

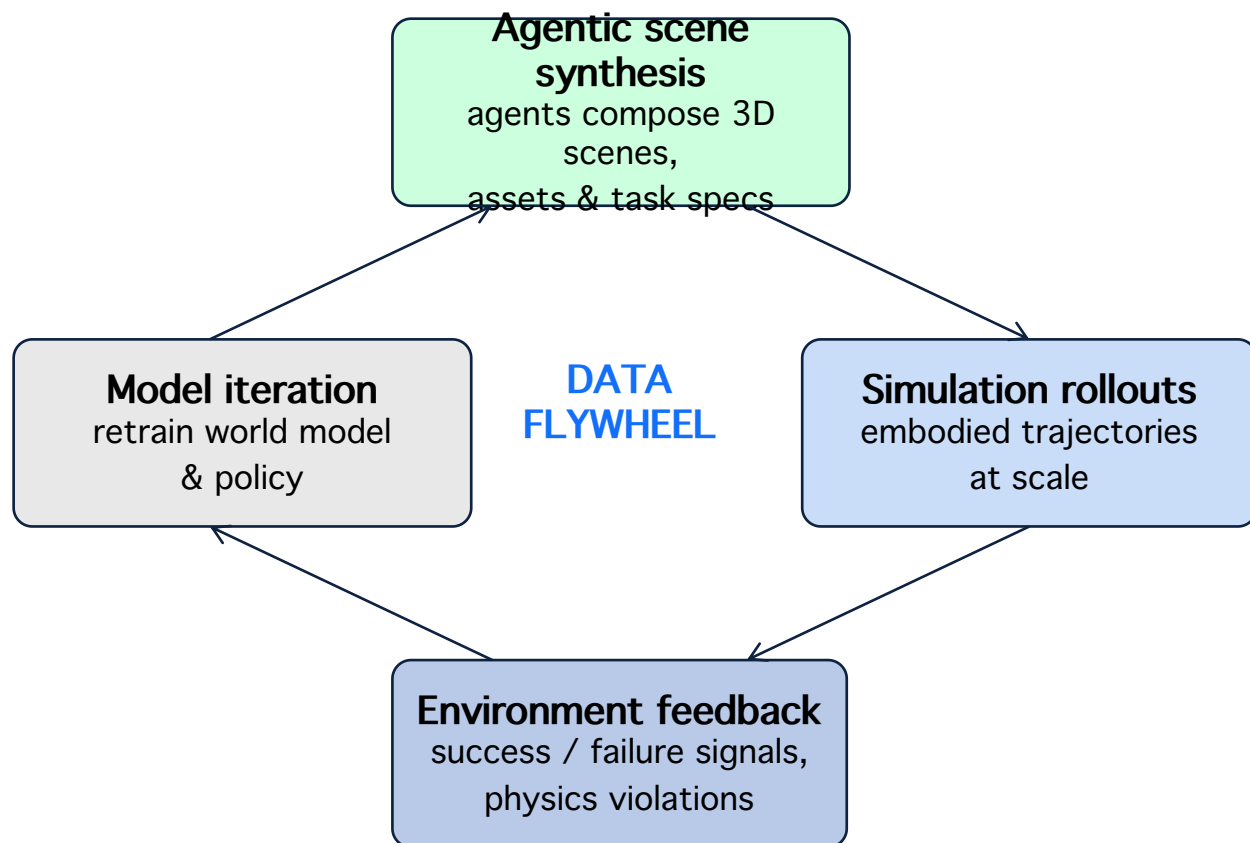
- 静态VLM Agent 只能从固定且有限的视角中获取信息，可用视角相互重复，或者和问题不匹配
- VLM 只能被动观察，无法主动探索

Method:

- 使用从粗到细的两阶段视角搜索策略
- 粗筛阶段过滤冗余视角
- 细搜阶段由Agent自主在3D场景中探索
- Agent 主动获取和问题关联的视觉信息

CoV 无需训练，将 VLM Agent 在 OpenEQA 测试上的表现提升11.98%，并展现出强大的Test-Time Scaling能力

■ 高效的 Agentic World Model



- 具身数据是核心瓶颈：真机采集慢，仿真场景依赖人工构建
- 核心思路：Agent 自动生成可仿真的场景与任务，并利用环境反馈进行筛选和优化
- 闭环迭代：每轮同时提升数据质量与模型能力

~100x cheaper

我们合成可仿真的场景的成本比商业场景构建流程低两个数量级。

Agentic World Model 将仿真转化为自我增强的数据飞轮，成本仅为商业方案的1%。

Part 3: Future Work

Sparse vs. Linear Attention

维度	Sparse	Linear
① 前缀缓存命中	✓ 语义无损，可截断可分叉	✗ 只能按块快照
② KV / 状态体积	✗ $O(n)$ ，全量 KV 不能丢	✓ $O(1)$ 固定状态
③ Attention FLOPs	✗ $O(n \cdot k)$ + 残余二次项	✓ 约低一个数量级
④ 长程精确召回	✓ 全量 KV 均可被选中	✗ 状态容量有界
⑤ 推理栈成熟度	✓ 复用现有 paged KV	✗ 状态管理仍在补齐

整个训练、推理pipeline的scaling up以及成本才是王道!

Mixed Attention is promising for multimodal inference:

Sequence 维 — Global linear + Local sparse

按频率切同一序列：低频全局走 linear 抽象，高频局部走 sparse 精确交互

Depth 维 — SANA-Video 2.0 [1]

按层切 (类比LLM)：m:1 交错，每 m-1 层一个 softmax anchor

Kernel 维 — HLA

[1] SANA-Video 2.0, arXiv 2607.21553.

■ 扩展 “World Model” 的规模

World Model 尚无统一的正式定义:

预测式 (Predictive) — 输出未来的潜在状态
Model-based RL: Dreamer (V1-V3), MuZero
JEPA 系列: I-JEPA, V-JEPA, V-JEPA 2, LeJEPA

生成式 (Generative) — 输出像素空间的观测
视频生成: Sora, Veo, Cosmos, Wan 3.0
交互式神经游戏引擎: Genie, GameNGen
驾驶场景生成: GAIA-1/2, Vista

模拟式 (Simulative) — 输出几何与物理状态
空间智能 / 3D World Model: World Labs Marble, HunyuanWorld
科学模拟器: GraphCast, MDGen

决策式 (Decision-making) — 输出动作
World Action Model (WAM): Lingbot-WM, Cosmos-Policy
Vision-language Action Model (VLA): Lingbot-VA, π 系列

语言 / 统一式 (Language / Unified)
LLM 作为 World Model: RAP, WebDreamer
统一理解与生成模型: Janus, Emu3, BAGEL

为什么需要 World Model?

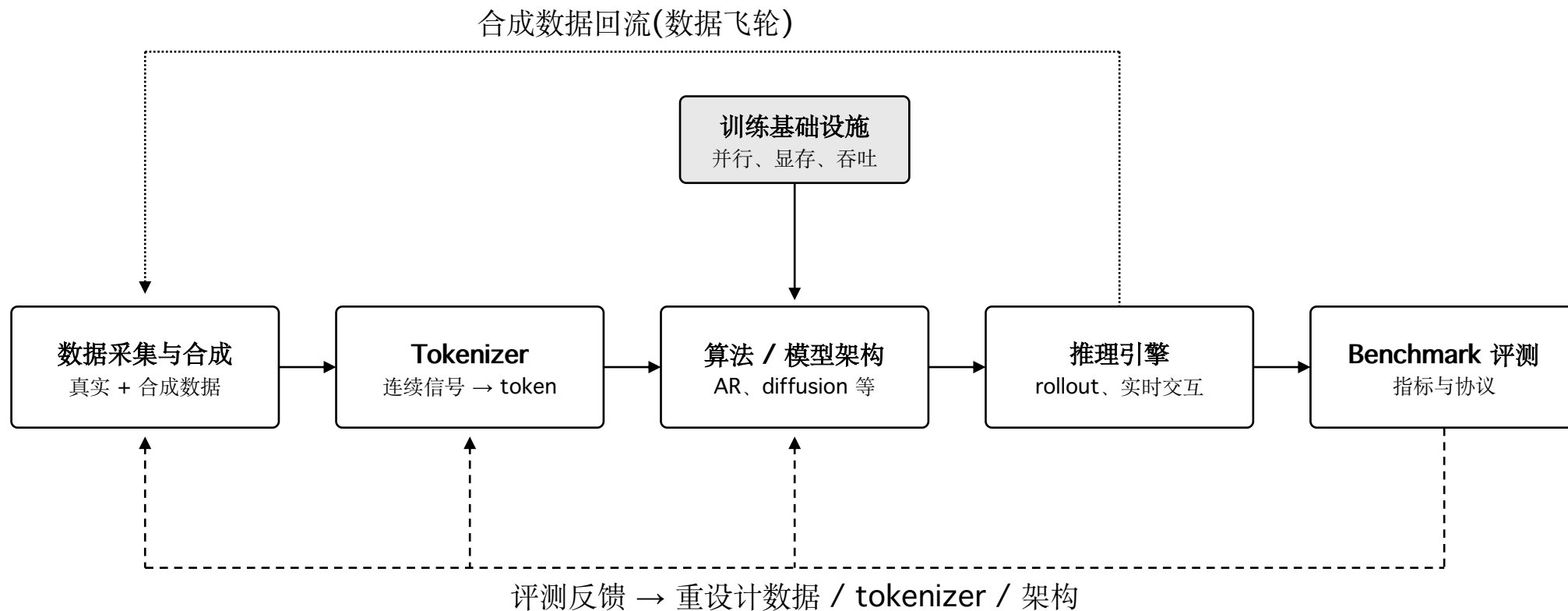
- 作为 Agentic AI、Embodied AI 和游戏的模拟器
- World Model Scaling up → 涌现更强的感知与推理能力

Real World Model 应具备:

- 实时
- 可交互
- 物理合理
- 3D 几何一致
- 长时生成
- 高保真

World Model Scaling Law: Pipeline 视角

Scaling 是各环节强耦合的 pipeline 协同设计问题



最大瓶颈依然在数据侧：质量 vs. 规模 —— 高质量数据难扩量，易得数据难保真

e.g., 具身场景的数据来源：真机遥操作、手持采集、动捕重定向、第一人称视频生成、Isaac仿真合成、跨本体聚合等

■ Agentic World Model 的 Serving Infra

Step 1 - World Model Serving

Harness for kernel / infra / hardware optimization

- 先搭 world model serving 的 harness: 上下文、工具、控制流、沙箱
- 由 Agent 完成 kernel 优化: 算子生成、调优、正确性回归
- 覆盖 infra 与 hardware: 并行策略、调度, 显存与带宽
- 用 profiling / benchmark 度量优化收益



Step 2 - Agentic World Model Serving

VLM for spatial reasoning + 更强的 harness 与推理 infra

- 引入 VLM 做 spatial reasoning, 赋予 world model 空间理解
- 改进 harness: 多模态上下文、空间记忆、任务编排
- 改进推理 infra: 视觉 token 压缩、状态复用、异步调度
- 形成完整的 agentic world model serving 栈

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



我们造的是一个负担得起思考、并且会自己变强的 agent。

如果还有领域也需要一个 DeepSeek 时刻，我们想成为
agentic multimodal generation 里的那一个。

Thank you!